

Application Security Engineering — Study Plan

Independent Self-Study — Kyle Miskell — kmiskell@protonmail.com

0

Pre-
Study

I

AppSec
Intro

II

Web &
Network

III

Core
Vulns

IV

Testing &
Tooling

V

Cloud
Security

VI

Code
Review

VII

Sec+
Cert

VIII

Interview
Prep

IX

First
Days

Contents

What Is Application Security?	3
Why AppSec Engineering	4
– The Case for This Path	
– Your Structural Advantage	
Why This Moment	5
Target Role: Application Security Engineer	6
– What It Is — Conceptually	
– What It Is — In Practice	
– Day-to-Day Responsibilities	
– Philadelphia & Remote Market	
Plan Schedule Overview	7
– Phase Timeline	
Study Methodology	8
– Local & Regional Events	
– Note-Taking	
– Portfolio: Writeups & Project Site	
AppSec: Potential Pitfalls & Risk Assessment	9
Study Plan Phase Summary	10
0 — Pre-Study: Foundations	12
– Learning Resources	
– Schedule	
– Projects & Writings	
– Concepts and Their Use on the Job	
I — AppSec Intro: A Dip into Offense	15
– Learning Resources	
– Schedule	
– Projects & Writings	
– Concepts and Their Use on the Job	
II — Web & Network Foundations	18
– Learning Resources	
– Schedule	
– Projects & Writings	
– Concepts and Their Use on the Job	
III — Core Vulnerability Classes	23
– Learning Resources	
– Schedule	
– Projects & Writings	
– Concepts and Their Use on the Job	
IV — Security Testing & Tooling	28
– Learning Resources	
– Schedule	
– Projects & Writings	
– Concepts and Their Use on the Job	
V — Cloud Security Fundamentals	36
– Learning Resources	
– Schedule	
– Projects & Writings	
– Concepts and Their Use on the Job	
VI — Secure Code Review & Threat Modeling	39

– Learning Resources	
– Schedule	
– Projects & Writings	
– Concepts and Their Use on the Job	
VII — CompTIA Security+ Certification	42
– Learning Resources	
– Schedule	
– Projects & Writings	
– Concepts and Their Use on the Job	
VIII — Interview Preparation & Interviewing	45
– Interview Preparation	
– Application Strategy	
IX — Expected First Days on the Job	48
– Your Immediate Differentiators	
Appendix	49
– Philadelphia AppSec Community	

What Is Application Security?

Application security — AppSec — is the discipline of finding, fixing, and preventing security vulnerabilities in software. Where most of software engineering asks *does this work?*, application security asks *can this be broken?* The question sounds simple. The implications are not.

Software has an attack surface: every input field, every API endpoint, every authentication flow, every dependency in your requirements.txt is a potential entry point for an adversary. AppSec engineers systematically map that surface, probe it for weaknesses, and work with development teams to close them — before attackers find them first.

The discipline spans the entire software development lifecycle. In the design phase, it means threat modeling — drawing data flows and asking what happens when an attacker controls each input. During development, it means secure code review — reading code the way an attacker reads it, looking for injection points, broken authentication, insecure data handling. In testing, it means static analysis (SAST) that reads source code for vulnerability patterns, dynamic analysis (DAST) that probes a running application, and manual penetration testing that combines both with adversarial creativity.

The OWASP Top 10 is the field's shared vocabulary for the most critical web application security risks. SQL injection, broken access control, cross-site scripting, security misconfigurations, software supply chain failures, and other key risks. These are not abstract academic concerns — they are the specific vulnerability classes that appear in real breaches against real companies, year after year. AppSec engineers know them the way a surgeon knows anatomy: not as a list to memorize, but as a framework for understanding how systems fail.

This plan targets web application security specifically — the attack surface that lives at the HTTP layer, in APIs, in authentication flows, in the browser.

Why AppSec Engineering

The Case for This Path

You spent five years building the attack surface that AppSec engineers protect. You know FastAPI microservices, React frontends, AWS infrastructure, Docker containers, legacy PHP codebases — the exact technology stack that AppSec work asks you to reason about adversarially.

Your Structural Advantage

Most AppSec engineers come from one of two directions: former developers who learned security, or security professionals who learned development. You are in the first category, which means you understand not just what the vulnerability is but why developers write it. The code patterns that lead to SQL injection, authentication bypasses, and CORS misconfigurations are patterns that developers encounter every day. Recognizing them as vulnerabilities — rather than just implementation choices — is the shift this plan builds. That background is the primary qualification.

Specific transferable skills from your resume:

- **FastAPI microservices** — REST API security is a core AppSec domain. Hands-on API security knowledge built throughout this plan means arriving on day one prepared for the work.
- **Python** — The primary scripting language for AppSec tooling. Custom Semgrep rules, vulnerability scanning automation, security utilities.
- **React frontends** — Client-side vulnerabilities (XSS, CSRF, insecure storage) require understanding how the browser executes JavaScript. React experience provides a strong foundation for understanding client-side vulnerability patterns.
- **AWS, Docker, Kubernetes** — Cloud and container security are AppSec-adjacent and increasingly part of the role. Your infrastructure knowledge is a differentiator.
- **Legacy PHP, raw SQL, and NoSQL** — You have worked in legacy PHP, hand-rolled MySQL, and DynamoDB — exactly where SQL injection, IDOR, and NoSQL injection live. You have seen where those patterns come from.
- **Architecture & domain modeling** — You designed microservices and domain-driven services with deliberate separation of concerns and trust boundaries. Threat modeling is that same structural thinking turned toward attack: mapping data flows and asking where trust is assumed.
- **Testing discipline** — You wrote unit, integration, and regression tests. AppSec testing follows the same deterministic logic: inputs, expected behavior, failure modes.

Why This Moment

The case for entering application security in 2026 rests on concrete numbers, not hype. Here is what the data actually shows.

The breach problem is not going away.

IBM's Cost of a Data Breach Report (2025 edition) — the most cited annual benchmark in the industry — found that the average U.S. cost of a data breach reached a record \$10.22 million in 2025, even as the global average fell slightly to \$4.44 million driven by faster AI-assisted detection. The U.S. number is the highest ever recorded. Nearly two-thirds of the 600 organizations IBM studied said they were still recovering from their breach at the time of the report. Verizon's 2025 Data Breach Investigations Report found that exploitation of vulnerabilities — the class of attack that AppSec engineers specifically exist to prevent — was present in 20% of all breaches studied; the 2026 edition reports that vulnerability exploitation has overtaken stolen credentials as the leading way attackers get in.

The workforce gap is structural.

ISC2's recent Workforce Studies estimate a global cybersecurity workforce gap of roughly 4.8 million people. The U.S. Bureau of Labor Statistics projects 29% job growth in information security analysis from 2024 to 2034 — nearly ten times the average growth rate across all occupations. For application security specifically, Mordor Intelligence reports a 15% annual shortfall of AppSec engineers in the United States — demand growing faster than the supply of qualified people to fill it.

The shortage is not evenly distributed. ISC2 noted in 2025 that skills shortages now outweigh headcount shortages as the primary constraint — meaning companies are not just short on bodies, they are short on people who can actually do the work. AI is automating the lowest complexity tasks (Tier 1 alert triage, routine vulnerability scanning), which is compressing demand for those roles. What it is not automating is the judgment-intensive work: finding business logic flaws that automated scanners cannot detect, threat modeling a new microservice architecture, explaining a critical finding to a VP of Engineering in terms that result in a fix. That work requires a human who can read code, understand systems, and think adversarially. That is the work this plan prepares you for.

The AppSec market is expanding fast.

Straits Research values the global application security market at \$12.71 billion in 2025 and projects \$42.29 billion by 2034, about 14% annual growth. Application security testing is one of its fastest-growing segments. Mid-level Application Security Engineer postings commonly list roughly \$130,000–\$180,000, and Product Security Engineer postings are among the faster-growing security titles.

The structural argument: IBM's data shows that organizations with severe security staffing shortages face \$1.76 million higher breach costs than those that are adequately staffed. Companies are not hiring AppSec engineers because it is a nice-to-have. They are hiring because the cost of not having them is measurable, large, and growing.

Target Role: Application Security Engineer

What It Is — Conceptually

Web application development and application security share the same foundation: HTTP, APIs, authentication flows, browser security models, database queries. Every endpoint in a web application is an AppSec concern. Every authentication flow is a potential bypass. Every React component that renders user input is an XSS candidate. Developers who learn AppSec understand the systems being defended — they are adding an adversarial lens to knowledge they already hold.

The shift is perspective. Where a developer asks *does this feature work correctly?*, an AppSec engineer asks *what happens when an attacker controls this input?* The underlying knowledge of the system is identical. The adversarial lens is the discipline.

What It Is — In Practice

- **Secure code review** (analogous to: pull request review, but adversarial) — Reading code to identify vulnerability patterns: SQL queries built by string concatenation, tokens stored in `localStorage`, endpoints missing authorization checks, user data rendered without sanitization.
- **Penetration testing / DAST** (analogous to: integration testing, but adversarial) — Running Burp Suite against a staging environment, intercepting HTTP requests, modifying parameters, observing whether the application behaves securely.
- **SAST tooling** (analogous to: adding a linter to CI/CD) — Integrating Semgrep into the pipeline. Writing custom rules that catch company-specific vulnerability patterns. Triage scan results: true positives from false positives.
- **Threat modeling** (analogous to: architecture review, but adversarial) — Given a proposed system design, enumerate the ways it could be attacked. STRIDE: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege.
- **Vulnerability management** (analogous to: bug triage) — Prioritizing findings by severity and business impact. Writing clear reports developers can act on. Tracking remediation. Verifying fixes.

Day-to-Day Responsibilities

- Secure code review on high-risk PRs and new features — auth, payments, file upload, API endpoints
- Running and maintaining SAST tooling in CI/CD — Semgrep rules, false positive triage, new rule development
- Manual penetration testing of web applications and APIs — Burp Suite, methodology-driven, OWASP-aligned
- Threat modeling new system designs — STRIDE analysis, data flow diagrams, risk documentation
- Vulnerability reporting and remediation tracking — clear writeups, severity ratings, developer liaison
- Security training and developer education — lunch-and-learns, secure coding guidelines, champions programs

- Dependency and supply chain security — SCA scanning, CVE monitoring, patch prioritization

Philadelphia & Remote Market

AppSec has strong remote availability — a large share of postings offer remote or hybrid flexibility. Local Philly demand is moderate but real, anchored by healthcare, financial services, and enterprise tech.

Start applying at Week 16 for AI-native startups and security-conscious tech companies. Target Philadelphia legal AI and healthtech in parallel — these companies hire locally and a full-stack background is immediately relevant.

Plan Schedule Overview

This plan is calibrated for 6–8 months at 25 study hours per week. 6 months is the optimistic case; 8 months is the outer bound; 7 months is the realistic midpoint. The hardest phase is not technical — it is mental: shifting from builder to adversarial thinker.

Day Type	Days/Week	Hours	Notes
Long Study	5	5 hrs	Reading, labs, implementation, tool practice, note card review, writeups
Day Off	2	—	No study, no work. Required for retention & sanity.

Phase Timeline

0	I	II	III	IV	V	VI	VII	VIII	IX
Pre-Study	AppSec Intro	Web & Network	Core Vulns	Testing & Tooling	Cloud Security	Code Review	Sec+ Cert	Interview Prep	First Days
Wks 1–3	Wk 4	Wks 5–8	Wks 9–15	Wks 16–22	Wk 23	Wks 24–26	Wks 27–30	Wks 31–34	Wk 35+

Timeline: 6 months is the optimistic case; 8 months is the outer bound; 7 months is the realistic midpoint. The first 1–3 weeks are pre-study before security content begins. Start applying at Week 16 for startups and AppSec-adjacent roles. Security+ lands on your resume at Week 30, strengthening the main application push that follows.

Study Methodology

The methods below apply to every phase. Following them consistently is the difference between completing labs and actually owning the material.

Local & Regional Events

Studying is not only reading and labs — attending local and regional security events is part of the work, both to rebuild the sense of being part of the industry after heads-down solo study and because referrals come from people who have met you. A few worth knowing: the OWASP Philadelphia chapter (the most directly relevant group), Philly 2600 (the lowest-pressure entry point), and OWASP Global AppSec USA (city changes yearly; 2026 is San Francisco). Start showing up early — even one finished PortSwigger lab is enough to bring and talk about. See the Appendix section “[Philadelphia AppSec Community](#)” for the full list of groups and conferences.

Note-Taking: Notes → Note Cards

Two outputs for every chapter or lab section: working notes during study, note cards derived from them afterward. Notes are for comprehension; note cards are for retention.

- **During study:** Detailed working notes in your own words. For security concepts: what the vulnerability is, why the code is exploitable, how an attacker uses it, how to fix it. Questions you cannot answer yet.
- **After each section:** Extract note cards. One concept per card. Front: vulnerability name. Back: what it is, how to identify it in code, how to fix it. If you cannot explain the fix without looking, you do not own the concept.
- **Everything in Phases II–V gets full note cards** — this is the material you will use daily on the job.

Portfolio: Writeups & Project Site

AppSec portfolios are built differently from software engineering portfolios. What distinguishes an AppSec candidate is documented offensive work: vulnerability writeups, lab solutions, CTF walkthroughs, threat model documents. A well-written writeup shows someone finding a vulnerability, understanding why it exists, exploiting it, and explaining the fix, in a way a resume line cannot.

- **Writeup format:** one format for every writeup, taught in Phase I step 04 and used from Phase III on: (1) finding, (2) risk rating with CVSS, (3) summary, (4) technical details and evidence, (5) impact, (6) remediation, (7) code-review pattern — what a reviewer looks for in source to catch this class. The GitHub version follows this structure; the LinkedIn version can be a narrative rewrite of it (800–1,200 words).
- **Target:** the thirteen writeups scheduled before Week 16 (Phases I–III), then the Juice Shop assessment report and the threat model before the main application push. Quality over quantity.
- **GitHub:** Commit from Day 1 — the very first commit is this study plan and its build script. Keep everything in public repositories — the plan, notes, tooling-phase scripts, custom Semgrep rules, automation tools, and lab artifacts — building a continuous commit history from the start. Every README explains the methodology and what the artifact demonstrates — not just usage instructions.

AppSec: Potential Pitfalls & Risk Assessment

AI Replacing AppSec Engineers — Real but Limited

AI-assisted SAST tools (GitHub Copilot Autofix, Snyk Code, Semgrep with AI triage) are improving and will automate more routine vulnerability scanning over time. The roles being automated are the lowest-end scan-and-report functions, not the manual code review, threat modeling, and developer consultation that constitutes senior AppSec work. Arriving with a portfolio of demonstrated manual security work is what automation cannot supply.

Entry-Level Hiring Compression

AI tools are handling some work that junior security analysts used to do. The mitigation is the same as it has always been in security hiring: arrive with a portfolio of real work rather than a list of certifications. Someone who can walk through a stored XSS chain and connect it to a real bypass they identified during code review stands out on portfolio alone.

Certifications Are Not the Portfolio

Sec+ is a recognized baseline credential in enterprise and regulated hiring. The plan treats Sec+ as the baseline and the portfolio as the evidence of skill.

Moderate Philly Market

Philadelphia's AppSec market is real but not as deep as New York, DC, or San Francisco. The mitigation is remote — a large share of AppSec postings offer remote or hybrid. Philadelphia is a base, not a constraint. The local network at OWASP and BSides matters most for referrals to companies not on job boards.

Scope Creep into Red Team

AppSec engineers frequently perform application-level penetration testing as part of the role — Burp Suite, manual web app testing, OWASP methodology. This is expected and built into the plan. The drift to avoid is into full red team or network penetration testing, which is a distinct specialization requiring different skills (Active Directory, network exploitation, infrastructure pivoting). Red team is typically a destination role requiring demonstrated AppSec or pentest experience first — not a direct entry point from developer. Master the AppSec role first.

Net assessment: The risks are real. None negate the market opportunity. The plan as designed — hands-on lab work over certifications, manual security skill over automated tooling familiarity, full-stack deployment capability — is structured to be resilient to the most likely downside scenarios.

Study Plan Phase Summary

This plan turns a working software engineer into an entry-level AppSec engineer over roughly six to eight months. Rather than front-loading theory, it follows a deliberate arc: shore up the engineering base, get hands dirty with offense early, then build depth and professional tooling on top of that foundation.

It opens by refreshing the fundamentals a developer already half-knows — Python and FastAPI, HTTP, Docker, a working local lab (Phase 0) — and immediately follows with a first taste of the attacker mindset (Phase I), so security feels concrete from the start rather than abstract. It then fills the web and network gaps most developers skipped on the way to shipping features (Phase II).

The middle of the plan is the heart of the offensive skill set: going deep on the core vulnerability classes — injection, XSS, broken authentication and access control, SSRF, and the rest — by exploiting each one hands-on in deliberately vulnerable labs (Phase III), then layering on the professional toolchain (Burp Suite, plus SAST, DAST, dependency and secrets scanning) and earning the Burp Suite Certified Practitioner certification, a hands-on web-exploitation credential (Phase IV). A light thread of AI/LLM security — prompt injection, output handling, tool-calling risk — runs through these phases as well.

The back half pivots toward the work an AppSec engineer actually does day to day — cloud security basics (Phase V) and secure code review and threat modeling (Phase VI) — rounded out by CompTIA Security+ for breadth and as the baseline credential enterprise and regulated employers recognize (Phase VII).

It closes with focused interview preparation (Phase VIII) and the job search itself (Phase IX). Applications begin in parallel from Week 16, though — well before the curriculum is finished — because a growing portfolio of writeups, with BSCP preparation under way, is what actually opens doors.

Phase 0 — Pre-Study: Foundations (Wks 1–3). **Purpose:** quickly refresh and expand the engineering base before security content — Python/FastAPI, HTTP, Docker, a working lab. **Resources:** FastAPI docs, HTTP (Notes + MDN), Docker (Notes). **Achievement:** a complete FastAPI app built from scratch and a running local lab.

Phase I — AppSec Intro: A Dip into Offense (Wk 4). **Purpose:** first taste of the attacker mindset — how web vulnerabilities are found and exploited. **Resources:** *Real-World Bug Hunting* (Ch 1–5), PortSwigger Web Security Academy, TryHackMe, OWASP WebGoat. **Achievement:** first hands-on exploitation (WebGoat and PortSwigger) and the SQL injection essay published.

Phase II — Web & Network Foundations (Wks 5–8). **Purpose:** deep HTTP, web, and authentication foundations plus Burp basics. **Resources:** *Alice and Bob Learn Application Security*, WAHH, Burp Suite docs, PortSwigger Authentication path. **Achievement:** Burp fluency, HTTP/auth/session mastery, and the PortSwigger Authentication, JWT, and OAuth labs cleared.

Phase III — Core Vulnerability Classes (Wks 9–15). **Purpose:** master the core web vulnerability classes hands-on at practitioner depth. **Resources:** PortSwigger topic paths and their apprentice/practitioner labs, *Hacking APIs*, the OWASP Top 10, and the OWASP Top 10 for LLM Applications. **Achievement:** practitioner labs across the OWASP classes and the OWASP LLM Top 10, plus an OWASP Top 10 writeup series.

Phase IV — Security Testing & Tooling (Wks 16–22). **Purpose:** tooling fluency — Burp mastery, SAST, DAST, SCA, and secrets scanning — and the BSCP certification. **Resources:** Burp docs, PortSwigger BSCP practice exams, Semgrep, Bandit, ZAP, Snyk, TruffleHog, Gitleaks. **Achievement:** passed the Burp Suite

Certified Practitioner (BSCP) exam, completed a full Juice Shop security assessment, and secured a small LLM-backed API.

Phase V — Cloud Security Fundamentals (Wk 23). **Purpose:** AWS security basics — shared responsibility, IAM, and common misconfigurations. **Resources:** flaws.cloud, flaws2.cloud, AWS IAM docs. **Achievement:** every level of the flaws.cloud and flaws2.cloud AWS security challenges completed, with one combined writeup, and the Phase 0 container image scanned and hardened.

Phase VI — Secure Code Review & Threat Modeling (Wks 24–26). **Purpose:** the core AppSec craft — manual secure code review and threat modeling. **Resources:** OWASP Code Review Guide, OWASP ASVS, *Threat Modeling: Designing for Security*. **Achievement:** a documented code review and a threat model project.

Phase VII — CompTIA Security+ Certification (Wks 27–30). **Purpose:** earn the baseline credential recognized in enterprise and regulated hiring; main application push. **Resources:** the *Get Certified Get Ahead: SY0-701* study guide and Professor Messer practice exams. **Achievement:** passed CompTIA Security+ (SY0-701).

Phase VIII — Interview Preparation & Interviewing (Wks 31–34). **Purpose:** convert skills and portfolio into offers — interview prep, story framing, application strategy. **Resources:** *Impractical Python Projects*, a cybersecurity interview question bank, GitHub Security Lab, and mock interview platforms. **Achievement:** a polished portfolio and interview narrative, plus active interviewing.

Phase IX — Expected First Days on the Job (Wk 35+). **Purpose:** start strong in the new sub-field and role and know your immediate differentiators. **Resources:** your accumulated notes and reference books. **Achievement:** a first-two-weeks plan, a clear grasp of your immediate differentiators, and early on-the-job wins.

0

Pre-Study: Foundations

Weeks 1–3 · Before Security Content Begins

Three parallel tracks before security content begins. The FastAPI track is the main one — 1–3 weeks to close out the official docs and build a complete project from scratch. The others are a few days each, run in parallel. All of this is active, hands-on work. By the end of Week 3, AppSec content starts immediately.

Learning Resources

- **FastAPI Official Documentation** — Official docs, self-contained. Read start to finish.
- **Existing HTTP Notes** — Personal notes (HTML intro + Odin HTTP). Cover request-response, methods, status codes, URL anatomy, headers, DNS, TCP/IP basics.
- **MDN Web Docs — Using HTTP Cookies** — Specifically for Secure, HttpOnly, and SameSite cookie attributes. The one HTTP topic not covered in existing notes.
- **Existing Docker Notes** — Personal notes and note cards covering containers, images, Dockerfiles, networking, volumes, Compose, Swarm, and Stack. More comprehensive than any online tutorial.
- **DigitalOcean — Install and Use Docker** — a step-by-step, command-by-command walkthrough of the container lifecycle.
- **OWASP WebGoat** — Deliberately insecure web application covering OWASP Top 10 vulnerability classes with structured hands-on exploitation modules. Runs via Docker.

Pre-Study Schedule — Weeks 1–3

01. SETUP: Git & GitHub Hygiene · Day 1

- GitHub profile already exists with past projects. Before new portfolio work begins: confirm profile is public, bio is current, location is set to Philadelphia, PA.
- Review existing pinned repos — anything from the web dev years that still reflects well is worth keeping pinned. Anything that does not, unpin.
- Create a new repository for the FastAPI project. Commit as you build — clean messages, logical history. This repo is a portfolio artifact from day one and will be the most recent, actively developed project on the profile.
- Set up LinkedIn in parallel: headline and About section state the deliberate transition into application security as of August 2026. Certifications and portfolio links get added as they are earned and built.

02. REVIEW: HTTP & Cookies · Days 1–3

- **HTTP:** Review existing HTTP notes (PDF, .odt, and note cards). These cover the request-response model, methods (GET, POST, PUT, PATCH, DELETE), status codes (2xx, 3xx, 4xx, 5xx), URL anatomy, headers, DNS, and TCP/IP basics.

- **Cookies:** Cookies are not covered in the existing notes. Read [MDN 'Using HTTP cookies'](#) specifically for the security attributes: Secure (only sent over HTTPS), HttpOnly (inaccessible to JavaScript), and SameSite (controls cross-site request behavior). These three attributes appear throughout security review work.
- **SQL:** SQL is strong from prior study — joins, normalization, indexing. Do a fast review using existing SQL note cards.

03. READ + BUILD: FastAPI Documentation — Complete · Weeks 1–3

- [The official docs](#) cover Dependencies, Security, Middleware, CORS, Request Body, and Response Model — exactly the sections most relevant to AppSec work. Finishing the docs builds the complete mental model needed to audit production web codebases.
- **Starting point:** Start from the beginning. Re-read your notes from the covered sections (intro, path params, query params) first, then continue through the remaining tutorial sections. Slow down at Dependencies, Security, Middleware, and CORS — those four sections warrant the most detailed notes.
- **Build a personal FastAPI project as part of the reading:** Five years of adding to others' APIs is solid experience. Building one end-to-end — from blank file to deployed container — completes the picture. Design something small but complete: 5–6 endpoints, JWT authentication implemented via FastAPI's Security module, at least one dependency-injected auth check, a SQL database connection. The goal is to own a codebase fully from the ground up, which makes it the best candidate for security analysis work later in the plan.

04. LEARN + PRACTICE: Docker · Days 2–4

- Review your existing Docker notes and note cards to refresh the fundamentals — containers and images, the run / stop / remove lifecycle, and port mapping.
- Work through DigitalOcean's [How to Install and Use Docker](#) guide for a command-by-command walkthrough of the container lifecycle: pull an image, run a container, list with `docker ps`, then stop and remove it.
- **Practice:** pull and run [OWASP WebGoat](#) in a container — publish it to a port, open it in the browser, then stop and remove it. Standing up a vulnerable target app in a container is the Docker skill you will actually use as an AppSec engineer, and you will repeat it for Juice Shop and other target apps later.

Projects & Writings — Pre-Study

Project: FastAPI API — Built From Scratch

A complete FastAPI application: 5–6 endpoints, JWT auth, dependency-injected auth checks, SQL database. Committed to GitHub with clean history. First portfolio artifact.

Project: Dockerized FastAPI App

Dockerfile for the FastAPI project. Built into an image, run as a container.

Concepts and Their Use on the Job

FastAPI security internals

How FastAPI implements authentication and authorization via the Security module and Depends(). Where JWT validation happens. How middleware intercepts requests. How CORS is configured at the framework level. **On the job:** These are the layers you audit when reviewing application code for security issues.

Cookie security attributes

Secure, HttpOnly, and SameSite are not just configuration options — they are security controls. **On the job:** Their absence or misconfiguration is a finding in any security review.

Docker as a tool in AppSec

Containers provide isolation but also create attack surfaces: exposed ports, over-permissioned images, secrets in environment variables. **On the job:** Standing up a vulnerable target app in a container, as with WebGoat here, is how you run every target environment in this plan and most assessment work.

I

AppSec Intro: A Dip into Offense

Week 4

From outside, AppSec can look inaccessible without years of CTF experience. It is not, and this phase exists to demonstrate that directly — before any systematic study begins.

The goal is not to become a penetration tester in one week. It is to arrive at the Web & Network Foundations phase having touched real vulnerability classes with your hands, developed genuine questions about why they work the way they do, and established the foundational mental model: every input is a potential attack vector, and your job is to think about what happens when an attacker controls it.

Learning Resources

- **PortSwigger Web Security Academy** — The most widely recommended AppSec learning resource among practitioners. Interactive labs, structured learning paths, every major vulnerability class covered. Built by the team that makes Burp Suite. Primary hands-on platform for the entire plan.
- **OWASP WebGoat** — A deliberately insecure application for learning. Runs locally via Docker. Structured modules per vulnerability class.
- **TryHackMe** — Browser-based, beginner-friendly, guided rooms. The ‘Jr Penetration Tester’ path covers [Burp Suite: The Basics](#), web vulnerabilities, and penetration testing methodology. [HTTP in Detail](#) is a standalone room.
- **“Real-World Bug Hunting”** — Peter Yaworski, No Starch Press, 2019 — A highly accessible introduction to web vulnerabilities. Real bug bounty reports, plain English, organized by vulnerability class.

AppSec Intro — Week 4

01. EXPLORE: TryHackMe Orientation · Week 4

- Complete two short [TryHackMe](#) rooms — [HTTP in Detail](#) and [Burp Suite: The Basics](#)
- These are short and designed for orientation, not mastery

02. EXPLORE: PortSwigger Academy Orientation · Week 4

- [Create a free PortSwigger account](#)
- Skim the [SQL injection](#) material to get oriented, then complete the first 3–4 Apprentice & Practitioner level [labs](#)
- Skim the [cross-site scripting](#) material to get oriented, then complete the first 2 Apprentice-level [labs](#) (reflected and stored)
- Reading the official solution is completely fine in this phase. Phase I is orientation, not mastery — the goal is to see how an attack actually works and lose the sense that it is magic, not to invent it unaided. If a lab stumps you, follow the walkthrough. You will learn these vulnerability classes properly, and solve them without help, in Phases II–III.

03. READ: “*Real-World Bug Hunting*” Ch. 1–5 · Week 4

- [Chapter 1](#) covers bug bounty and HTTP basics; Chapters 2–5 are each a vulnerability class — open redirect, HTTP parameter pollution, CSRF, and HTML injection / content spoofing — illustrated with real disclosed bug bounty reports
- Read actively: when a report describes a finding in a Python or REST API context, pause and think about what the vulnerable pattern looks like in code — this builds the adversarial mental model

04. LEARN: Writing Pentest Reports · Week 4

- Complete the free TryHackMe room [Vulnerabilities 101](#) (about 20 minutes). The goal is Task 3, Scoring Vulnerabilities (CVSS & VPR): how a CVSS score is built from its metrics
- Complete the free TryHackMe room [Writing Pentest Reports](#) — its write-up structure has eight parts: title, risk rating, summary, background, technical details and evidence, impact, remediation advice, references
- The plan’s writeup format keeps six of those (the finding as title, CVSS as the risk rating, summary, technical details and evidence, impact, remediation) and adds one the room does not cover — the **code-review pattern**: two to four lines on what a reviewer looks for in source to catch this class (the vulnerable construct, the safe construct that replaces it, and where it tends to hide). Sources for it: the ‘How to prevent’ section of the PortSwigger topic page and the class’s [OWASP Cheat Sheet](#)
- Practise the full seven-part format once on a WebGoat lesson you have finished (SQL Injection intro is the easiest) so the format is familiar before Phase III
- First portfolio use of the format is the Phase III vulnerability-class writeups; the WebGoat lessons in the next step are practice, logged as notes only

05. PROJECT: WebGoat — First Exploitation · Week 4

- Ensure local [WebGoat](#) still executes as expected in local Docker
- Complete these WebGoat lessons: SQL Injection (intro), Cross Site Scripting, Cross Site Scripting (stored), Authentication Bypasses, and Insecure Direct Object References (IDOR)
- For each lesson: exploit it, then keep a short notes-only log — the payload used, what it did, and one line on the root cause in the Java source. These are practice notes, not portfolio writeups: the lessons hand you the vulnerability, so a formal report would overstate the work
- Then write the essay ‘What SQL Injection Taught Me About Trusting Input’ (900–1,200 words): a plain-English account of what SQL injection is, why it works, and one thing that surprised you in the WebGoat and PortSwigger labs. Publish on LinkedIn when polished

Projects & Writings — AppSec Intro

Notes: WebGoat Practice Log

For each WebGoat lesson completed, a short entry in the notes repo: payload, effect, and root cause in the code. Practice notes, not a portfolio finding — the lessons hand you the vulnerability. The formal vulnerability-class writeups begin in Phase III; the essay below is the first published piece.

Writing: What SQL Injection Taught Me About Trusting Input (900–1,200 words)

Plain-English explanation of what SQL injection is, why it works, and one thing that surprised you. Publish on LinkedIn when polished. Framing: ‘I have been building APIs for five years. Here is what I learned when I tried to break one.’

Concepts and Their Use on the Job

The OWASP Top 10

Introduced as a taxonomy. You have seen SQL injection and XSS in practice. The full Top 10 is a structured catalog of the most critical web vulnerability classes. Mastery of each class comes through dedicated study. **On the job:** This is the vocabulary of every code review and architecture conversation. When a developer asks why their code is flagged, you explain which OWASP category it falls into and why it matters.

Injection as a class

Any time user input reaches an interpreter without proper sanitization, injection is possible. SQL injection is the canonical example. Command injection, LDAP injection, and template injection follow the same logic. **On the job:** When reviewing an API endpoint that accepts user input, your first question is always: does this input reach an interpreter? A database query? A shell command? A template renderer? This question catches vulnerabilities automated tools miss.

The attacker's mindset

The mental shift from 'does this work?' to 'what happens when I control this input?' This is the foundational perspective change. You have started making it.

HTTP as the attack surface

Every web vulnerability lives in the HTTP layer — parameters, headers, cookies, request bodies. Understanding HTTP is the substrate for every vulnerability class in this plan. **On the job:** Every Burp Suite session, every manual pentest, every DAST scan result is an HTTP transaction. Reading raw HTTP requests and responses fluently is required for the work.

II

Web & Network Foundations

Weeks 5–8

This phase builds the technical foundation for AppSec work. Every major vulnerability class lives inside HTTP, authentication protocols, browser security models, or API design patterns. The phase reframes these layers from an adversarial perspective — not just how they work, but why they fail. It closes with the defender's view of AppSec (Alice and Bob) and cryptography fundamentals.

Learning Resources

- **[“Alice and Bob Learn Application Security”](#)** — Tanya Janca (SheHacksPurple), Wiley, 2020 — Covers threat modeling, secure code review, SDLC security, and how to work with developers. Covers the defender-side, program-building angle that pure offensive resources miss.
- **[“The Web Application Hacker’s Handbook”](#)** — Dafydd Stuttard and Marcus Pinto, Wiley, 2011 — Foundational reference written by the creator of Burp Suite. Ch. 1–3 cover web application security, core defense mechanisms, and web application technologies. Ch. 6 covers authentication, Ch. 7 session management, Ch. 8 access controls. Some content is aging but these chapters remain the clearest treatment available.
- **[PortSwigger Web Security Academy](#)** — Primary lab platform for this phase. Covers authentication attacks, access control vulnerabilities, and SQL injection — all with interactive labs.
- **[MDN Web Docs](#)** — The authoritative reference for HTTP, cookies, CORS, CSP, and the browser security model. Covers each security header, its syntax, and its security purpose in depth.
- **[OWASP SSRF Prevention Cheat Sheet](#)** — Covers the two SSRF cases, allowlist and resolved-address validation, IMDSv2 on AWS, and deny-lists as a last resort.
- **[TryHackMe ‘Pre-Security’ path — Network Fundamentals module](#)** — Primary resource for the networking week. Covers OSI model, TCP/IP, ports, and how packets travel in about two interactive hours. Not Net+, not CCNA — precisely scoped for AppSec-relevant networking.
- **[TryHackMe ‘Pre-Security’ path — Putting It All Together room](#)** — Covers how all the components of a web request fit together: DNS, HTTP, load balancers, CDNs, databases, WAFs, and how each layer relates to security decisions.
- **[OWASP Cryptographic Storage Cheat Sheet](#)** — Primary reference for the cryptography week. Covers which algorithms are acceptable and which are broken, key management, and storage patterns.
- **[OWASP Transport Layer Security Cheat Sheet](#)** — TLS configuration, certificate management, and common misconfigurations. Companion to the crypto week.
- **[hashcat](#)** — Password cracking tool used in the cryptography week to crack an MD5 hash and make weak hashing concrete. The wiki’s Dictionary attack and Example hashes pages give the exact commands for an MD5 wordlist attack.
- **[bcrypt \(Python library\)](#)** — Python library for bcrypt password hashing. Used in the cryptography week to implement correct password hashing in Python. Simple API: hash a password, verify a password.

- **OpenID Connect — How Connect Works** — The OpenID Foundation's overview of how OIDC adds an identity layer on top of OAuth 2.0.
- **OWASP SAML Security Cheat Sheet** — What breaks in SAML single sign-on (signature wrapping, replay, weak validation) and how to review it.

Web & Network Foundations — Weeks 5–8

01. READ: HTTP Deep Dive · Week 5

- Read [WAHH](#) Ch. 1–3 (web application security, core defense mechanisms, web application technologies) as foundational context
- You installed Burp and learned its basics in Phase I. Now go deeper via the official documentation — [Proxy](#), [Repeater](#), and [Intruder](#) — the primary tools you will use throughout this phase
- Read [CORS misconfigurations](#), [CSP](#), [TLS](#), [HTTP methods](#), and [status codes](#) for their security implications
- **Full notes + note cards:** CORS misconfigurations, CSP, TLS, HTTP methods, and status codes with security implications

02. READ + LABS: Input Handling Fundamentals · Week 5

- Read the PortSwigger [SQL injection](#) learning path, then work the apprentice-level [labs](#) to internalize the input-validation root cause — the practitioner labs follow in Phase III
- **Full notes + note cards:** SQL injection root cause, parameterized queries, injection taxonomy

03. READ + LABS: Authentication & Session Management · Weeks 5–6

- Read [WAHH](#) Ch. 6 (Attacking Authentication) and Ch. 7 (Attacking Session Management)
- Complete the PortSwigger [Authentication](#) learning path — all apprentice and practitioner [labs](#)
- Complete the [JWT](#) learning path, then its [labs](#) — unverified signature, flawed signature verification, weak signing key, jwk / jku injection, and *algorithm confusion*; then the [OAuth](#) learning path and its [labs](#) — implicit-flow bypass, forced profile linking, and *account hijacking via redirect_uri*
- Read how [OpenID Connect](#) layers identity on OAuth 2.0 and the [OWASP SAML Security Cheat Sheet](#); no labs, one page of notes each. OAuth, OIDC, and SAML are the three protocols an authentication review meets
- **Full notes + note cards:** Password enumeration, session fixation, JWT algorithm confusion, OAuth redirect URI manipulation

04. READ + LABS: Access Control & APIs · Week 7

- Read [WAHH](#) Ch. 8 (Attacking Access Controls)
- Read the PortSwigger [Access control](#) and [API testing](#) learning paths, then work the apprentice-level Access control [labs](#) and API testing [labs](#) to build the foundational mental model — the practitioner labs follow in Phase III
- **Full notes + note cards:** IDOR, horizontal/vertical privilege escalation, mass assignment, excessive data exposure

05. PROJECT: Authentication Audit Checklist · Week 7

- Write a 1–2 page authentication audit checklist from scratch — no templates
- Every item should come from a concept in your notes
- For each item: what you check, how you check it (manually and with what tool), and what a finding looks like
- Run the checklist against the FastAPI app you built in Phase 0 — it implements JWT authentication via [FastAPI's Security module](#), so it is the ideal first audit target. Record any findings in the seven-part report format from Phase I step 04
- Push to GitHub as Markdown. First real security artifact in the portfolio.

06. STUDY: Networking Fundamentals for AppSec · Week 7

- Complete the TryHackMe 'Pre-Security' path — [Network Fundamentals module](#). Take notes.
- Review existing notes for the URL-to-response flow.
- Read the [OWASP SSRF Prevention Cheat Sheet](#), then the one-page 'DNS Rebinding' section in WAHH Ch. 13 for the mechanism the concept below relies on.
- Complete the TryHackMe 'Pre-Security' path — [Putting It All Together room](#).
- **Full notes + note cards:** OSI layers 3/4/7 with one AppSec example each; TCP vs UDP with one security implication each; the URL request flow; common ports; private IP ranges; what a firewall, load balancer, and reverse proxy each do.

07. WRITE: Authentication Writeup + LinkedIn Post · Week 7

- Publish 800–1,200 word LinkedIn article: 'Five ways JWT authentication breaks — including the one I found in my own API'
- Lead with the finding from your own authentication audit (step 05), then the four remaining flaws from the PortSwigger JWT labs. If the audit found nothing, use the five lab flaws and retitle to 'Five ways JWT authentication breaks — and what I look for in code now'
- Frame it as a developer who learned to break what they built

08. READ: "*Alice and Bob Learn Application Security*" · Week 8

- Read "[Alice and Bob Learn Application Security](#)" Ch. 1–5 and Ch. 7: security fundamentals, security requirements, secure design, secure code, and common pitfalls (Ch. 1–5), and what an AppSec program does day-to-day (Ch. 7)
- Read during this week — this is the defender and program-building perspective that PortSwigger and WAHH do not cover
- **Key insight:** AppSec is not just finding vulnerabilities — it is building processes so developers stop introducing them
- **Full notes + note cards:** SDLC security touchpoints, developer security hygiene checklist, what an AppSec program actually does day-to-day

09. STUDY: Cryptography Fundamentals · Week 8

- Review existing sys design notes on auth and basic security before any new reading. The goal this week is the AppSec angle: how cryptographic primitives fail, what broken implementations look like in code, and how to flag them in a security review.
- Read the [OWASP Cryptographic Storage Cheat Sheet](#) and the [OWASP Transport Layer Security Cheat Sheet](#). Take notes.
- **Hands-on:** [Crack an MD5 hash with hashcat](#). [Implement bcrypt password hashing in Python](#). These make the concepts concrete.
- **Full notes + note cards:** One card per algorithm type covered in the cheat sheets. Front: name and use case. Back: when to use it, what breaks it, what a broken implementation looks like in code.

Projects & Writings — Web & Network Foundations

Project: Authentication Audit Checklist

A 1–2 page checklist written from your own notes, run against the Phase 0 FastAPI app, findings recorded in the seven-part format. Pushed to GitHub as Markdown. First real security artifact in the portfolio.

Writing: 'Five ways JWT authentication breaks — including the one I found in my own API' (800–1,200 words)

LinkedIn article leading with the audit finding, then four flaws from the PortSwigger JWT labs. Framed as a developer who learned to break what they built.

Notes: full notes and note cards

HTTP security headers, authentication and session attacks, access control, input validation, cryptography, and networking for AppSec — the Phase II card deck.

Concepts and Their Use on the Job

HTTP security model

Every HTTP header that has security implications, what it does, and what happens when it is misconfigured. The substrate of every web vulnerability. **On the job:** When reviewing a PR that sets cookies or configures CORS, you check HttpOnly, Secure, SameSite, and Access-Control-Allow-Origin as a reflex.

Authentication and session security

How authentication protocols work and how they fail. JWT algorithm confusion is a recurring real-world finding.

On the job: JWT configuration is the first thing you check: algorithm whitelisted? Signature verified? Claims validated? Three questions catch the most common JWT vulnerabilities.

Access control

IDOR, privilege escalation, horizontal vs. vertical access control flaws. Ranked A01 in the OWASP Top 10:2025. **On the job:** When reviewing an API endpoint that returns data based on a user-supplied ID, you ask: is the user's authorization checked against the requested resource, or just their authentication? This question catches most IDOR.

API security

REST API-specific vulnerability patterns — broken object-level authorization (IDOR), mass assignment, and excessive data exposure — and why APIs widen the attack surface beyond traditional server-rendered applications.

Input validation

The root cause of injection vulnerabilities. Every injection class shares the same root cause: unsanitized user input reaching an interpreter.

Cryptography fundamentals

Hashing vs. encryption vs. encoding. Which algorithms are broken (MD5, SHA-1 for passwords, DES, RC4) and which are correct (bcrypt, AES-GCM, RSA). How to identify weak crypto in a code review without being a cryptographer. **On the job:** When you see a password hash, you check whether it is bcrypt/Argon2 (correct) or MD5/SHA-1 (flag immediately). When you see encrypted data, you check for AES-GCM (correct) vs AES-ECB (flag). When you see a secret, you check whether it is hardcoded or pulled from a secret manager. These are reflex checks that take seconds once the vocabulary is owned.

Networking fundamentals for AppSec

OSI layers 3 (Network/IP), 4 (Transport/TCP-UDP), and 7 (Application/HTTP) and why AppSec lives at layer 7. TCP vs UDP and one security implication of each. The steps of a URL request from DNS query to HTTP response. Common ports and the services on them. Private IP ranges (10.x, 172.16–172.31.x, 192.168.x), localhost (127.0.0.1), and the AWS metadata endpoint (169.254.169.254). Standard web app network architecture: internet → firewall → load balancer → app server → database. What a firewall, load balancer, and reverse proxy each do and what security question to ask about each. **On the job:** When asked ‘where does TLS terminate?’ in an architecture review, you can answer and explain the security implication. When reviewing a deployment configuration, you check for unexpected open ports. When a threat model shows a service making outbound HTTP calls, you ask whether SSRF mitigations are in place and whether private IP ranges are blocked. When a design review describes traffic from an EC2 instance to 169.254.169.254 and asks if it is concerning, the answer is yes — SSRF against the metadata endpoint.

Networking as an attack surface

DNS rebinding exploits the gap between DNS validation and the TCP connection, bypassing SSRF protections. SSRF against private IP ranges and the AWS metadata endpoint (169.254.169.254) is a common high-severity finding in cloud-hosted applications. Where TLS terminates in an architecture tells you whether internal traffic is encrypted; HTTP statelessness is why cookies exist and therefore an attack surface; and unexpected open ports in a deployment configuration are a finding. These are the specific networking behaviors AppSec engineers encounter in real work, not networking-engineering concepts.

AppSec program thinking (Janca)

AppSec is not just finding bugs — it is building the processes, training, and SDLC integration that stop bugs from being introduced. The developer’s perspective on the defensive side of the role.

III

Core Vulnerability Classes

Weeks 9–15

Phase III is the technical core of the plan. The OWASP Top 10 is the field's shared vocabulary for the most critical web application security risks. Every code review uses them as a checklist. Every penetration test methodology is organized around them. This phase goes deep on each class — not survey knowledge, but genuine understanding of root cause, exploitation mechanics, and remediation. Complete the labs each step names; expert labs are stretch goals.

Learning Resources

- [PortSwigger Web Security Academy](#) — Primary lab platform for this phase. Interactive labs covering every major vulnerability class: injection, XSS, CSRF, SSRF, XXE, access control, business logic, deserialization, file upload, path traversal, and API testing.
- [OWASP Top 10 \(Web\)](#) — The ten most critical web application vulnerabilities, 2025 edition. Each entry includes a description, example attack scenarios, and prevention guidance.
- [OWASP API Security Top 10](#) — The API-specific companion to the web Top 10. Covers BOLA (Broken Object Level Authorization), broken authentication, BOPLA (Broken Object Property Level Authorization, which merges the older mass assignment and excessive data exposure categories), BFLA (Broken Function Level Authorization), and SSRF as they manifest in REST APIs.
- [OWASP Testing Guide v4.2](#) — The methodology reference. For each vulnerability class, it describes how to test systematically.
- [“Alice and Bob Learn Application Security”](#) — Tanya Janca (SheHacksPurple), Wiley, 2020 — Ch. 6 (testing and deployment) and Ch. 8 (securing modern applications and systems: APIs, microservices, serverless) are read in step 06. Ch. 1–5 and 7 were read in Phase II.
- [“Hacking APIs”](#) — Corey Ball, No Starch Press, 2022 — REST API security testing. Ch. 1–3 and Ch. 6–9 cover API reconnaissance, endpoint discovery, and authentication testing.
- [OWASP Top 10 for LLM Applications](#) — the standard risk list for LLM-backed apps: prompt injection, improper output handling, excessive agency, and sensitive information disclosure.
- [Semgrep](#) — Open-source SAST tool. First used here to write one rule for the SQL injection pattern exploited in step 01; the full SAST work is in Phase IV. ([rule-writing docs](#))

Core Vulnerability Classes — Weeks 9–15

01. READ + LABS: Injection Classes: Weeks 9–10

- **Buy Burp Suite Professional now** (~\$499/yr) — the out-of-band labs in this phase need [Burp Collaborator](#), which is Pro-only. Buying at the start of Phase III lets you do every lab here in place instead of skipping the Collaborator-based ones, and you use Pro continuously from here through the BSCP exam. See [Burp Suite Professional](#).

- Re-read your Phase II [SQL injection](#) notes (the learning path stays linked if you need it), then complete the practitioner [labs](#) — the UNION-based and blind (boolean- and time-based) techniques, plus the out-of-band labs (Burp Collaborator — you now have Pro)
- Work through the PortSwigger [OS command injection](#) and [Server-side template injection](#) learning paths, then their labs — the OS command injection [labs](#) (the simple case plus blind time-delay and output-redirection, and out-of-band exfiltration via Collaborator) and the basic SSTI [labs](#) only (enough to recognize SSTI in review; skip the engine-specific gadget exploitation)
- **Defense side of the same bug:** install the [Semgrep](#) CLI and [write one rule](#) that catches the SQL injection pattern you just exploited (a string-built query reaching a cursor); run it against your Phase 0 app and a WebGoat source file. This is your first custom rule; the Phase IV one is the second
- **For each lab:** exploit it, then keep the same notes-only log as the WebGoat step — the payload used, what it did, and one line on the root cause. The portfolio writeups for this phase are the Top 10 series and the SSRF piece in step 02

02. PROJECT: OWASP Top 10 Writeup Series: Throughout Weeks 9–15

- One 800–1,200 word writeup per [OWASP Top 10](#) category, published progressively throughout this phase
- Not academic summaries — each in the standard seven-part report format from Phase I step 04, plus a real-world example. Use the 2025 edition. For the four categories with lab evidence from Phases II–III (A01 Broken Access Control, which now includes SSRF, A04 Cryptographic Failures, A05 Injection, A07 Authentication Failures) the evidence is your lab exploitation; for the six without one (A02 Security Misconfiguration, A03 Software Supply Chain Failures, A06 Insecure Design, A08 Software or Data Integrity Failures, A09 Security Logging and Alerting Failures, A10 Mishandling of Exceptional Conditions) the evidence is a documented real-world case (for example Log4Shell for A03, the 2021 Codecov breach for A08) and the code-review pattern carries the weight
- Write an eleventh, standalone SSRF piece from the step 04 labs in the same format — SSRF lost its own Top 10 slot in 2025 but remains a distinct class with its own exploitation mechanics and defenses
- These eleven writeups are the backbone of your AppSec portfolio

03. READ + LABS: Cross-Site Scripting: Week 11

- Complete the PortSwigger [Cross-site scripting](#) learning path, then the [labs](#) covering reflected, stored, and DOM XSS, the DOM-sink variants, and the exploitation labs (Exploiting cross-site scripting to steal cookies, Exploiting XSS to bypass CSRF defenses); skip the long tail of near-identical context-encoding and filter-bypass permutations
- **XSS in React:** React's JSX escaping prevents many reflected XSS vectors by default. The key exception is [dangerouslySetInnerHTML](#), which bypasses JSX escaping entirely and is a standard code review finding.
- **Key:** XSS to account takeover chains — cookie theft, CSRF token theft, keylogging

04. READ + LABS: CSRF, SSRF, XXE: Week 11

- Complete the PortSwigger [CSRF](#), [SSRF](#), and [XXE](#) learning paths, then their labs — the CSRF [labs](#) in full (each is a distinct broken-defense pattern), the apprentice and practitioner SSRF [labs](#) (skip the two expert labs — Shellshock and the whitelist-filter bypass), and the core XXE [labs](#) (file retrieval, SSRF, and one blind example — all apprentice and practitioner)
- **SSRF:** API endpoints that fetch user-supplied URLs are the primary SSRF surface

- [Cloud metadata service](#) attacks (169.254.169.254) are the canonical SSRF exploitation target

05. READ + LABS: Access Control & Business Logic: Week 12

- Review your Phase II [Access control](#) notes, then complete the remaining practitioner [labs](#) — the URL-, method-, and Referer-based bypasses and multi-step gaps
- Work through the PortSwigger [Business logic vulnerabilities](#) learning path, then its apprentice and practitioner [labs](#) — each is a distinct logic flaw, and this is the class your developer background most directly advantages, so cover it well
- Business logic is the vulnerability class automated tools miss entirely — developer background is the structural advantage here

06. READ + LABS: API Security Deep Dive: Weeks 12–15

- Read the [OWASP API Security Top 10](#) in full
- Key additions over the web Top 10: BOLA (Broken Object Level Authorization = IDOR for APIs), BFLA (Broken Function Level Authorization), and BOPLA, the 2023 merge of mass assignment and excessive data exposure
- Read “[Hacking APIs](#)” Ch. 1–3 (how web applications and APIs work, and the common API vulnerability classes) and Ch. 6–9 (API discovery and reconnaissance, endpoint analysis, authentication attacks, and fuzzing)
- Read “[Alice and Bob Learn Application Security](#)” Ch. 6 (testing and deployment) and Ch. 8 (securing modern applications and systems — APIs, microservices, serverless); the chapters not covered in Phase II
- Review your Phase II [API testing](#) notes, then complete the remaining [labs](#): [mass assignment](#), [the unused endpoint](#), and both server-side parameter pollution labs ([query string](#), practitioner; [REST URL](#), expert-level)
- Work through it treating your own past API work as the mental target: would the APIs you built have been vulnerable?

07. READ + LABS: Deserialization, File Upload, Path Traversal: Week 13

- Complete the PortSwigger [Insecure deserialization](#), [File upload](#), and [Path traversal](#) learning paths, then their labs — the deserialization [labs](#) concept set only (modifying serialized data, using application functionality, one gadget-chain example; skip the extra language-specific gadget chains), the apprentice and practitioner file upload [labs](#) in full, including the expert web-shell-via-race-condition lab (solved with the free Turbo Intruder extension), and the path traversal [labs](#) in full (each is a distinct filter bypass)
- Less common than injection and XSS but high-severity in enterprise codebases when present

08. READ + LABS: Web LLM Attacks (PortSwigger): Week 15

- Securing LLM-backed features is a fast-growing AppSec differentiator, and it builds directly on your API background
- [Web LLM attacks](#) learning path, then the [labs](#) — Exploiting LLM APIs with excessive agency, Exploiting vulnerabilities in LLM APIs, Indirect prompt injection, and Exploiting insecure output handling in LLMs

09. READ: OWASP Top 10 for LLM Applications: Week 15

- Now that you have exploited the web-facing LLM classes hands-on, read the [OWASP Top 10 for LLM Applications](#) straight through as the framework that names and organizes them
- Map what you did to the list, in order: prompt injection (LLM01), sensitive information disclosure (LLM02), improper output handling (LLM05), and excessive agency (LLM06) are the classes you attacked; the rest (supply chain, data and model poisoning, system prompt leakage, vector and embedding weaknesses, misinformation, and unbounded consumption) are the broader categories to recognize

Projects & Writings — Core Vulnerability Classes

Writing: OWASP Top 10 Writeup Series (ten pieces, 800–1,200 words each)

One seven-part writeup per 2025 category, published progressively through the phase; lab evidence for four categories, a documented real-world case for six. The backbone of the portfolio.

Writing: Standalone SSRF writeup

An eleventh piece in the same format from the SSRF labs — the class kept its own writeup after leaving the 2025 Top 10.

Notes: injection lab log

Payload, effect, and root cause for every SQL, OS command, and template injection lab. Practice notes, not portfolio pieces.

Concepts and Their Use on the Job

Injection — complete taxonomy

SQL, command, template, LDAP, NoSQL. Root cause: unsanitized user input reaching an interpreter. Fix: parameterized queries. **On the job:** Every database query, every shell command, every template render gets the same question: does user input reach this without sanitization? One question catches the whole class.

XSS — all three types

Reflected, stored, DOM. Root cause: user input rendered as HTML without encoding. Fix: context-appropriate output encoding. **On the job:** React escapes JSX output by default, which prevents many reflected XSS vectors. The three things to check in a React code review: dangerouslySetInnerHTML usage (bypasses JSX escaping entirely), third-party component XSS vectors, and any server-rendered template output that feeds into the React tree.

CSRF and SSRF

CSRF exploits browser trust. SSRF exploits server trust to reach internal resources. Fix: SameSite cookies for CSRF; URL allowlists for SSRF. **On the job:** Any endpoint accepting a URL parameter is flagged immediately. Does it fetch that URL server-side? Is the response returned to the user? Is there a URL allowlist? Three questions.

Access control flaws

The most common finding in production AppSec work. Fix: verify the authenticated user is authorized for the specific resource on every request.

Business logic vulnerabilities

Not detectable by automated tools. Requires understanding intended behavior. The developer background is the advantage. **On the job:** This is the vulnerability class that threat modeling catches before it ships. STRIDE analysis of a data flow diagram surfaces the business logic assumptions that could be violated.

Securing LLM-backed features

The LLM-specific classes — prompt injection, improper output handling, excessive agency, and sensitive information disclosure — sit on top of the classic vulnerabilities you already review, and map onto skills you have: injection is injection, output handling is untrusted-input handling, excessive agency is least privilege. **On the job:** An increasingly common AppSec task; review an LLM-backed endpoint with the same questions you ask of any endpoint that takes untrusted input and calls tools.

IV

Security Testing & Tooling

Weeks 16–22

Phase IV moves from knowing vulnerability classes to operating the professional toolchain used to find them systematically. Burp Suite is the primary tool. SAST (static analysis of source code, via Semgrep) and DAST (dynamic testing of the running app) are the automation layer. The goal is to reach a level of tool fluency where tooling becomes leverage, not a learning distraction — you use Burp Suite to find vulnerabilities faster, not to compensate for not understanding them.

Learning Resources

- **Burp Suite Documentation** — Official documentation for all Burp Suite tools: [Proxy](#), [Repeater](#), [Intruder](#), [Sequencer](#), and [Scanner](#).
- **Semgrep Documentation** — Primary for general SAST. A widely used open-source SAST tool. Sufficient for the entire plan.
- **Bandit Documentation** — Python-specific SAST tool. Detects insecure subprocess calls, `eval()` usage, weak SSL contexts, hardcoded passwords, insecure pickle and yaml usage.
- **GitHub code scanning (CodeQL)** — GitHub's built-in SAST, powered by CodeQL, enabled per repository with a default setup. Used once in this plan as the second SAST engine next to Semgrep. [CodeQL query docs](#).
- **DVWA (Damn Vulnerable Web Application)** — Deliberately insecure PHP/MariaDB practice application, run locally via Docker Compose (localhost:4280). The second DAST target alongside WebGoat; set up in step 04 and scanned with ZAP in step 05.
- **ZAP Documentation** — the free, open-source DAST tool (formerly an OWASP project): an intercepting proxy plus an automated scanner that crawls a running app and runs active and passive checks for common web vulnerabilities. A free alternative to Burp Scanner, with a baseline scan that wires easily into CI.
- **Snyk Documentation** — Software composition analysis (SCA) — finding vulnerabilities in third-party dependencies. Available with a free tier.
- **Syft and CycloneDX** — Syft generates a software bill of materials (SBOM) from an image or source tree; CycloneDX is the SBOM standard it emits. Used once in this plan, in the Snyk step. ([CycloneDX](#))
- **PortSwigger Web Security Academy** — Primary resource for Burp Suite mastery and BSCP preparation. The Burp Suite learning path and BSCP practice exams live here. The BSCP exam is practical and hands-on, not multiple-choice: two applications, three stages each (user account, admin interface, read /home/carlos/secret), in four hours. The practice exam is one application in two hours, half the real format, and a pass is direct evidence of hands-on web exploitation skill.
- **Phase 0 FastAPI app** — Your own repository — the SAST scan target for Semgrep's Python ruleset and Bandit (Bandit is Python-only), and the codebase your custom Semgrep rule is written against.
- **OWASP WebGoat** — Deliberately insecure Java application. Used here only as a running DAST target for ZAP.

- **OWASP Juice Shop** — Deliberately insecure modern web application. The primary target for this phase's full security assessment: Burp Suite manual testing, ZAP DAST scan, Semgrep SAST, Snyk SCA. Runs via Docker or Node.
- **Pwning OWASP Juice Shop** — The official companion guide. Groups every Juice Shop challenge by vulnerability class and has full solutions in its appendix; the structure for the Phase IV full assessment.
- **TruffleHog** — Secrets scanner that searches full git history for leaked credentials, including credentials committed and later deleted.
- **Gitleaks** — Lightweight secrets scanner. Runs as a pre-commit hook and CI gate. Scans the repo history by default; as a pre-commit hook it scans only the staged diff, fast enough for every PR.

Security Testing & Tooling — Weeks 16–22

01. LEARN + LABS: Burp Suite Mastery: Week 16

- Proxy, Repeater, and Intruder are already yours from Phase II — review those notes; no need to re-read the docs
- The two Community-edition tools you haven't used: **Decoder** (encode/decode/hash inline) and **Comparer** (visual diff of two messages)
- **Decoder**: [SQL injection with filter bypass via XML encoding \(SQL injection\)](#) — encode the payload as XML/HTML entities to bypass the WAF
- **Comparer**: [Username enumeration via different responses \(Authentication\)](#) — spot the one login response that differs
- **Both**: [Blind SQL injection with conditional responses \(SQL injection\)](#) — read the admin password one true/false answer at a time
- **By end of Week 16**: fluent with Decoder and Comparer, and comfortable with Proxy match-and-replace and Repeater tab groups

02. LEARN + LABS: Semgrep & Bandit — SAST Tooling: Week 16

- Semgrep is already installed from Phase III. [Run semgrep scan with its default Registry rules](#) and [Bandit](#) against your own Phase 0 FastAPI app — a tool run and a results triage, not code work on the app
- [Triage the results](#): distinguish true positives from false positives
- [Write a second custom Semgrep rule](#) targeting a different Python injection pattern — for example, an endpoint passing request parameters directly to a database query
- [Integrate Semgrep into a GitHub Actions workflow](#) (sample repo is fine)
- Enable [GitHub code scanning \(CodeQL default setup\)](#) on the FastAPI repo, let it run once, and read the results next to Semgrep's: same codebase, two engines, note what each catches that the other does not. [CodeQL](#), [SonarQube](#), and [Semgrep](#) are the SAST names that recur in postings; knowing one deeply and one by contact is enough
- **Bandit**: install bandit with pip. [Run against a sample Python project](#) and against some code in old projects. Look for: B101 (assert usage), B102 (exec usage), B105/B106/B107 (hardcoded password), B201 (Flask debug mode), B301 (pickle), B501–B504 (SSL/TLS issues). The finding IDs become vocabulary you use in code review conversations

- Run both tools against the same Python codebase and compare results. Bandit catches Python-specific antipatterns that Semgrep misses without custom rules. Semgrep catches logic-level patterns that Bandit's fixed ruleset cannot express. Together with the Gitleaks and Snyk gates this is a DevSecOps pipeline: every pull request is scanned, and a failing security check blocks the merge.

03. LEARN: Secrets Scanning — TruffleHog & Gitleaks: Week 16

- Hardcoded secrets (API keys, passwords, tokens committed to repos) are one of the most common real-world breach vectors. Several well-known breaches began with a credential committed to a repository
- **TruffleHog**: install and run against a sample repo. Uses hundreds of credential detectors and verifies each hit against the issuing service, so results are live secrets rather than regex noise. Can scan full git history, not just current code
- **Gitleaks**: install and run as a CI **pre-commit hook**. Lightweight and fast, well-suited for blocking commits before they land
- Integrate both into a **GitHub Actions workflow**: Gitleaks on every pull request (scanning only the changed lines), TruffleHog for scheduled full-history scans
- **In code review**: you are now looking for hardcoded strings that look like keys, passwords, or tokens — in source code, config files, Dockerfiles, and environment variable defaults
- **Full notes + note cards**: When to use TruffleHog vs Gitleaks, what a false positive looks like, what to do when a live secret is found (rotation procedure, not just deletion)

04. SETUP: DVWA — Second DAST Target: Week 16

- **DVWA** (Damn Vulnerable Web Application) is a second deliberately insecure practice app, written in PHP — same purpose as WebGoat, different codebase. Scanning two targets in the next step shows tooling work across more than one application
- Install via Docker Compose from the official repository: **clone digininja/DVWA**, change into the folder, run **docker compose up -d**. It serves on **http://localhost:4280** (not port 80). Give the containers a moment to start
- First run: log in with the **default credentials** (admin / password), open **Setup DVWA** from the menu and click **Create / Reset Database**. Set the security level to **Low** under DVWA Security so the scanner in the next step has findings to report
- Confirm both practice apps are reachable side by side: WebGoat (from Phase 0) and DVWA. Budget 15–30 minutes; the **troubleshooting section** covers the common start-up errors

05. LEARN: ZAP — DAST Basics: Week 16

- Run **ZAP** against both local practice apps — WebGoat (Phase 0) and DVWA (previous step) — and compare the two reports
- Understand the difference between DAST (testing a running application) and SAST (testing source code)
- **Triage ZAP results**: true positive, false positive, needs investigation

06. APPLY: Begin Job Applications: Week 16 onward

- With Phases I–III complete and thirteen published writeups, begin applying now — do not wait for curriculum completion; applications ramp into the main push at Week 30
- Targets, profile, and framing: see “[Application Strategy](#)” in Phase VIII

07. DOCUMENT: Save Exam-Ready Proof-of-Concept Exploits: Throughout Weeks 16–20

- PortSwigger publishes full PoC / solution code for every lab. As you clear each practitioner lab this phase, save its working exploit into one personal, searchable notes file — the BSCP is open to your own notes (see [PortSwigger’s prep guidance](#)).
- Digital notes are ideal — the exam is fully open-book (your own notes, the web, third-party tools, and Burp extensions are all allowed) and you sit it in your own browser alongside Burp, so keep everything in one searchable file (Markdown or plain text) or a notes app rather than on paper. Make payloads copy-paste-ready, and pre-load your payload lists and useful extensions into Burp before you start.
- Pre-adapt the high-value PoCs for exam conditions: turn XSS print()/alert() payloads into cookie-stealers, parameterize your SQLi and SSRF payloads, and keep Collaborator / OOB templates ready to paste.
- Organize them by vulnerability class so you can pull the right template in seconds under exam time pressure

08. LABS: Out-of-Band (OOB) Exploitation with Burp Collaborator: Week 17

- **Requires [Burp Suite Professional](#)** — Burp Collaborator is Pro-only, and nearly every blind / out-of-band variant on the BSCP routes through it. You first used Collaborator in Phase III (Pro from the start); this step is a focused OOB drill to lock in exam fluency — re-confirm each one fast.
- **Blind SQL injection:** re-read your notes on [blind SQL injection](#) (the learning path stays linked), then the out-of-band [labs](#) — *Blind SQL injection with out-of-band interaction* and *Blind SQL injection with out-of-band data exfiltration* (practitioner)
- **Blind XXE:** re-read your notes on [blind XXE](#) (the learning path stays linked), then the out-of-band [labs](#) — *Blind XXE with out-of-band interaction* and *Exploiting blind XXE to exfiltrate data using a malicious external DTD*
- **Blind OS command injection:** from the [OS command injection](#) learning path, the out-of-band [labs](#) — *Blind OS command injection with out-of-band interaction* and *... with out-of-band data exfiltration*
- Carry the same Collaborator technique into blind SSTI detection and any asynchronous sink — fire a DNS/HTTP callback to confirm the bug, then exfiltrate data through it
- **For each lab:** exploit it, then save the Collaborator payload and working exploit into your exam notes

09. LEARN: Snyk — Dependency Scanning: Week 17

- Install and authenticate the [Snyk CLI](#), then run `snyk test` in a sample Python project with known-vulnerable dependencies (for example an outdated requirements.txt) to produce a software-composition report listing each flagged package, its CVE, and the safe upgrade
- Understand what SCA (software composition analysis) is and what it does not catch
- Log4Shell was a dependency vulnerability, not an application vulnerability — SCA catches these
- Wire `snyk test` into CI alongside Semgrep and Gitleaks so newly disclosed dependency CVEs fail the build — the remediation is almost always a version bump

- Generate an SBOM for the FastAPI app with [Syft](#) ([CycloneDX](#) format) and feed it to *snyc test* or Gype; an SBOM is the inventory a supply-chain review starts from

10. PROJECT: Secure a Small LLM-Backed API: Week 17

- Add one small LLM-backed endpoint to your Phase 0 [FastAPI](#) app — a /summarize or support-reply route that calls a hosted model API (for example the [Anthropic](#) or OpenAI API). This is the flagship AI portfolio piece; it builds directly on your API background
- Before attacking it, sketch a one-page [STRIDE](#) pass on the new endpoint: the trust boundaries between user, model, and any tool the model can call. A warm-up for the full threat model in Phase VI
- Run the same three attack classes you practised in your [Phase III Web LLM Attacks step](#) against your own endpoint: prompt injection ([LLM01](#)); improper output handling ([LLM05](#)) — show the XSS or SQL injection that trusting model output enables, then fix it with encoding and parameterization; and excessive agency ([LLM06](#)) — least privilege plus an approval step for any tool the model can call
- Write it up in your standard report format and push it to GitHub; label each finding with its [OWASP LLM Top 10](#) ID (LLM01, LLM05, and so on). One focused 'securing an LLM API' writeup is a strong, current portfolio piece
- Scope: this is securing an LLM *application*, not machine learning — stay on the AppSec slice

11. READ + LABS: Remaining BSCP Topics: Weeks 17–18

- **Topics the exam can draw from that earlier phases skipped — close them before sitting the exam.** For each: read the learning path, then do the apprentice and practitioner labs.
- [HTTP request smuggling](#) learning path, then the [labs](#) — *basic CL.TE* and *basic TE.CL*, *obfuscating the TE header*, and *Exploiting HTTP request smuggling to capture other users' requests* (a BSCP favorite)
- [HTTP Host header attacks](#) learning path, then the [labs](#) — *basic password reset poisoning*, *Host header authentication bypass*, *web cache poisoning via ambiguous requests*, and *routing-based SSRF*
- [Web cache poisoning](#) learning path, then the [labs](#) — *unkeyed header*, *unkeyed cookie*, *multiple headers*, and *targeted poisoning using an unknown header*

12. LEARN + LABS: Targeted Scanning & Content Discovery: Week 18

- **Requires [Burp Suite Professional](#)** (the Scanner and Engagement tools are Pro-only). The exam is time-boxed, so you scan smart, not broad — this is an explicit BSCP skill.
- Read [using Burp Scanner during manual testing](#), then the [labs](#) — *Discovering vulnerabilities quickly with targeted scanning* and *Scanning non-standard data structures*. Right-click a single request → *Do active scan* to audit just that request in seconds instead of crawling the whole site.
- **Content discovery:** use Burp's [Engagement tools](#) → *Discover content* to enumerate hidden directories, files, and parameters (for example a hidden /admin) — a recurring first move on exam apps. Practice it on labs you have already solved.
- **Build the habit:** hand every new request to the Scanner while you test manually, and run content discovery before assuming an app's surface is fully mapped

13. DRILL: PortSwigger's Recommended Reinforcement Labs: Week 18

- Beyond one practitioner lab per topic, [PortSwigger recommends 8 specific skill-reinforcement labs](#) before the exam. Make sure each is solid — several are already covered in earlier steps; this is the consolidated checklist.
- **XSS:** *Exploiting cross-site scripting to steal cookies* — from [Cross-site scripting](#), the [labs](#); exfiltrate a real session cookie (not just an alert()), which is what the exam rewards (reinforces Phase III step 03)
- **Authentication:** *Brute-forcing a stay-logged-in cookie* — from [Authentication](#), the [labs](#); decode the cookie in Decoder, then brute-force it with Intruder
- **SSRF:** *SSRF with blacklist-based input filter* — from [SSRF](#), the [labs](#); the practitioner filter-bypass variant from Phase III step 04 ([lab](#)); re-confirm it cold
- Already done elsewhere — re-confirm you can solve each cold: *Forced OAuth profile linking* (Phase II), *Exploiting HTTP request smuggling to capture other users' requests* (step 11), *Blind SQL injection with out-of-band data exfiltration* (step 08), *SQL injection with filter bypass via XML encoding* (step 01), and *Discovering vulnerabilities quickly with targeted scanning* (step 12)
- Save the working PoC for each into your exam notes — you may refer to your own notes during the exam

14. PROJECT: Burp Suite Certified Practitioner Prep: Weeks 18–19

- Requires [Burp Suite Professional](#) (bought at the start of Phase III); the [BSCP exam](#) itself is ~\$99 per attempt
- Pass the [BSCP practice exam](#) in under 2 hours — and do it more than once — before booking the real exam. It is one application in two hours, half the real exam, so consistently passing it under time is the strongest signal you are ready

15. DRILL: Fluency — Re-Solve Cold: Week 19

- For each major class, take one practitioner lab you have already solved and re-solve it cold — no notes, no solution — in 15 minutes or less.
- Work through roughly 20 labs this way (about one per class). If a lab runs past 15 minutes or you reach for your notes, that class is not yet exam-fluent — drill it again.
- This is the gap between having seen a vulnerability and being able to exploit it under time pressure with no hints — which is exactly what the exam tests

16. DRILL: Mystery Labs — Cold Recognition: Week 19

- The exam gives you no vulnerability hints — you must recognize the class from behavior alone. The [mystery lab challenge](#) spins up a random lab with its title and description hidden, the closest practice for that.
- Solve at least 5 mystery labs cold — no hints, working only from recon — before booking the exam. Time each one; the goal is fast, confident classification.
- If a class keeps stumping you, go back to that topic's labs and notes, then return to the mystery challenge

17. DRILL: Exam Stamina — Six Labs in One Block: Week 19

- [The real exam](#) is a single continuous 4-hour session — two applications, three stages each, and a pass needs every stage. Rehearse that endurance before you book it.
- Take six practitioner labs you have not memorized and solve them back-to-back in one timed 4-hour block — own notes only, Burp Pro, Collaborator live, no long breaks.
- Track time per lab and practice triage: if one runs past budget, bank partial progress, move on, and circle back — the time management is as much the test as the exploitation

18. PROJECT: Full Security Assessment of Juice Shop: Weeks 19–21

- Conduct a full security assessment of [OWASP Juice Shop](#) using: Burp Suite (manual), ZAP (DAST), Semgrep with its JavaScript/TypeScript rulesets (SAST), Snyk (dependencies), TruffleHog/Gitleaks (secrets)
- Work through it systematically using the official companion guide [Pwning OWASP Juice Shop](#), which groups every challenge by vulnerability class — injection, broken authentication, XSS, broken access control, and more — so you can map each finding to an OWASP category; its appendix has full step-by-step solutions if you get stuck
- Reuse the systematic methodology and the seven-part report format from Phase I step 04, as used in your Phase III vulnerability-class writeups — this project is where the full workflow comes together: recon, tooling, manual exploitation, false-positive triage, and professional reporting
- Produce a written vulnerability report: executive summary, findings table (vuln, severity, CVSS, evidence, recommendation), detailed finding writeups
- Export the finished report as a polished, shareable standalone PDF — the document you can share with anyone who asks what your assessment work looks like
- This is the most important portfolio artifact in the plan — it demonstrates the complete AppSec workflow from tool setup to professional reporting

19. EXAM: Burp Suite Certified Practitioner: Weeks 20–22

- Take the [BSCP exam](#). The exam consists of two applications, each completed in three stages: access a user account, reach the admin interface, read /home/carlos/secret. 4 hours.
- If unsuccessful on the first attempt, review the topics where the attempt failed and retake. PortSwigger sets no limit on attempts (each attempt costs \$99).

Projects & Writings — Security Testing & Tooling

Project: Full Security Assessment of OWASP Juice Shop

Burp manual testing, ZAP, Semgrep, Snyk, and secrets scanning against one target, written up as a professional vulnerability report (executive summary, findings table, detailed findings) and exported as a standalone PDF. The most important artifact in the plan.

Project: Secure a Small LLM-Backed API

An LLM-backed endpoint added to the Phase 0 FastAPI app, attacked for prompt injection, improper output handling, and excessive agency, then fixed; writeup labelled with OWASP LLM Top 10 IDs.

Project: Custom Semgrep rule and CI security workflow

One custom Semgrep rule for a Python injection pattern, plus a GitHub Actions workflow running Semgrep, Gitleaks, TruffleHog, and Snyk.

Notes: exam proof-of-concept file

Every working practitioner-lab exploit, organised by vulnerability class and copy-paste-ready for the open-book BSCP exam.

Certification: Burp Suite Certified Practitioner

Passed exam; the credential that proves hands-on web-exploitation skill.

Concepts and Their Use on the Job

Burp Suite as an extension of manual thinking

Burp Suite intercepts and replays HTTP traffic. It does not find vulnerabilities — it makes manual exploitation efficient. Understanding of what to look for drives the tool, not the reverse. **On the job:** Nearly every web pentest engagement runs through Burp Suite or an equivalent proxy. Every time you need to understand what a web application is actually sending and receiving, it is open. The AppSec engineer's equivalent of a developer's debugger.

SAST vs. DAST vs. SCA vs. Secrets Scanning

Four distinct scanning approaches that catch different vulnerability classes. SAST reads code, misses runtime behavior. DAST tests a running app, misses code paths not exercised. SCA finds known CVEs in dependencies. Secrets scanning finds exposed credentials in code and history. A real AppSec program uses all four — plus Python-specific tools like Bandit for Python shops. **On the job:** Integrating Semgrep into a company's CI/CD pipeline, writing rules for company-specific vulnerability patterns, and triaging results is a common AppSec responsibility. The custom rule you built is the portfolio demonstration.

False positive triage

Automated tools have noise. Distinguishing real findings from false positives without dismissing everything is where security judgment matters more than tool knowledge.

Professional vulnerability reporting

Severity rating (CVSS), evidence documentation, clear remediation guidance. The output of AppSec work is a finding that a developer can act on — not just a list of things that are broken. **On the job:** Every engagement produces a report. The format you learned in the Juice Shop assessment is the format used in production. Clear findings, actionable remediations, evidence-backed.

Secrets management

Hardcoded credentials are a separate vulnerability class from application logic flaws. The fix is not deletion from current code — it is rotation of the exposed credential plus integration of a secret manager (HashiCorp Vault, AWS Secrets Manager) and pre-commit scanning to prevent recurrence. **On the job:** Gitleaks runs as a pre-commit hook and a CI gate. TruffleHog runs on a schedule against full git history. When a developer accidentally commits an API key, you have a runbook: alert the developer, rotate the credential immediately (assume it is already compromised), add the pattern to the blocklist. This is day-one operational work at many companies.

v

Cloud Security Fundamentals

Week 23

Modern AppSec work happens in the cloud. S3 misconfiguration and IAM over-permissioning are among the most frequently reported cloud misconfigurations, and credible threat modeling requires understanding the cloud attack surface an application runs on. This phase covers the AppSec-relevant subset — not Terraform or multi-cloud architecture, but enough to identify misconfigurations in code and architecture reviews, understand SSRF chains that target cloud metadata, and discuss findings with infrastructure teams.

Learning Resources

- **AWS Security CTF: Attacker Path** — Six levels covering real-world AWS misconfigurations: S3 bucket exposure, credential leakage, IAM misuse, and SSRF via the EC2 metadata service. Each level explains the vulnerability and its fix.
- **AWS Security CTF: Attacker & Defender** — Companion to [flaws.cloud](#) with two separate paths: attacker path (exploit misconfigurations in Lambda and ECS Fargate) and defender path (incident response on the same app: analyze the attack logs with jq and Athena).
- **AWS Shared Responsibility Model** — Official AWS documentation. Defines what AWS secures (infrastructure) versus what the customer secures (IAM, S3 policies, application code, data encryption).
- **AWS IAM Documentation** — Official AWS documentation. The IAM concepts section covers users, roles, groups, policies, and the principle of least privilege.
- **FreeCodeCamp Cloud Security Fundamentals in AWS** — Beginner-friendly guide covering IAM users and policies, S3 bucket configurations, MFA, and the AWS Shared Responsibility Model with hands-on examples.
- **Trivy** — Open-source scanner for container images: known vulnerabilities in OS and language packages, Dockerfile misconfigurations, and secrets baked into layers. Used to scan the Phase 0 FastAPI image.
- **OWASP Docker Security Cheat Sheet** — The hardening checklist for images and containers: non-root users, pinned base images, capability drops, secrets handling, read-only filesystems.
- **OWASP Kubernetes Security Cheat Sheet** — Read once for the vocabulary (pods, RBAC, network policies); no cluster work in this plan.

Cloud Security Fundamentals — Week 23

01. READ: AWS Foundations: Shared Responsibility & IAM: Week 23

- Read the [AWS Shared Responsibility Model](#) page completely. Write a one-paragraph summary in your own words: what AWS is responsible for, what you are responsible for, and what the boundary looks like for a web application running on EC2 behind an S3 bucket
- Read the IAM concepts in the [AWS IAM documentation](#): what a user is, what a role is, what a policy is, how they attach, what least privilege means in practice

- Read the [FreeCodeCamp Cloud Security Fundamentals in AWS](#) guide — beginner-friendly, hands-on, covers IAM and S3 configuration with real console examples
- **Full notes + note cards:** AWS Shared Responsibility boundary; IAM user vs. role vs. group; what a policy document looks like (JSON structure, Effect/Action/Resource); what least privilege means and why wildcard permissions (s3:*) are a finding

02. LABS: [flaws.cloud](#) — Levels 1–6: Week 23

- Create a free AWS account if you do not have one. The CTF runs against real AWS infrastructure — you need the [AWS CLI installed and configured](#) with a free-tier account
- [Complete all six levels](#). Exploit each level fully before reading the solution
- **After all six levels:** one combined writeup in the standard seven-part report format from Phase I step 04, one finding per level; the code-review pattern becomes a configuration-review checklist (IAM policies, bucket policies, instance metadata settings)

03. LABS: [flaws2.cloud](#) — Attacker & Defender Paths: Week 23

- **Two distinct paths:** attacker (exploit serverless and container misconfigurations) and defender (investigate the same attack from the logs).
- [Complete both paths](#). The defender path is the AppSec-relevant half — it teaches you to read what an attack left behind in the logs, which is how a finding is confirmed on the job
- **Key concepts reinforced:** IAM role scoping for Lambda and ECS tasks, container metadata credential exposure, and log analysis with CloudTrail, jq, and Athena

04. LEARN + PRACTICE: Container Security Basics: Week 23

- Read the [OWASP Docker Security Cheat Sheet](#), then scan the image of your Phase 0 FastAPI app with [Trivy](#) (image vulnerabilities, misconfigurations, exposed secrets) and triage the findings the same way as the SAST results in Phase IV
- Fix one finding in the Dockerfile (pin the base image, drop a capability, or confirm the non-root user) and rebuild; keep the before/after scan output in the notes repo
- Skim the [OWASP Kubernetes Security Cheat Sheet](#) once for the vocabulary (pods, RBAC, network policies); no cluster work in this plan
- **Full notes + note cards:** image scanning vs runtime security; least privilege in containers; where secrets must not live (image layers, environment defaults); IMDS access from a container

Projects & Writings — Cloud Security Fundamentals

Writing: [flaws.cloud](#) assessment writeup

One combined seven-part writeup covering all six levels, one finding per level, with a configuration-review checklist for IAM policies, bucket policies, and instance metadata settings.

Notes: AWS security note cards

Shared responsibility boundary, IAM users vs roles vs policies, S3 public-access controls, IMDSv2, and the flaws2 defender-path log analysis.

Concepts and Their Use on the Job

AWS Shared Responsibility Model

AWS secures the physical infrastructure. You secure what runs on it: IAM configurations, S3 policies, application code, data encryption. Misunderstanding this boundary is the root cause of many cloud breaches.

IAM — Identity and Access Management

Users, roles, groups, and policies. A role is what an AWS service or EC2 instance assumes to perform actions. A policy is a JSON document defining what actions are allowed or denied on which resources. Least privilege means a role should have exactly the permissions it needs and nothing more. Over-permissioned roles are the most common IAM finding. **On the job:** When a team presents a new service design, you ask: what IAM role does this service run as? What permissions does that role have? Could an attacker compromise this service and use its IAM role to access something they should not? Reading an IAM policy JSON, an S3 bucket policy, or a Lambda configuration and spotting the problem is the AppSec-specific cloud skill: auditing infrastructure, not writing it.

S3 bucket misconfiguration

S3 buckets store data. Public read access means anyone on the internet can read the contents. Authenticated AWS user access means any AWS account holder worldwide can read it. Both are common misconfigurations. Bucket policies, ACLs, and Block Public Access settings all interact — understanding which one controls what is the skill. **On the job:** When reviewing a deployment configuration or an IaC pull request, you check: is Block Public Access enabled at the bucket level? Are there wildcard permissions? Are access logs enabled on buckets containing sensitive data? These are 5-minute checks that catch high-severity findings.

EC2 instance metadata service (IMDS)

169.254.169.254 is accessible from within an EC2 instance and returns IAM credentials, instance ID, and configuration. SSRF against this endpoint gives an attacker the instance's IAM credentials. IMDSv2 requires a token header that prevents simple SSRF exploitation. Whether IMDSv2 is enforced is a standard AppSec checklist item. **On the job:** When you see an endpoint that accepts a URL and fetches it server-side, your first question is now: can this reach 169.254.169.254? Is IMDSv2 enforced on the EC2 instances this service runs on? Is there a URL allowlist? [flaws.cloud](#) Level 5 is the exact attack chain you are looking for.

Credential exposure in git history

AWS keys committed to a git repository — even if deleted in a later commit — are permanently visible in history. TruffleHog scans history. The fix is to rotate the exposed credential, not just delete it from current code, plus pre-commit scanning to prevent recurrence. **On the job:** When reviewing any code that configures AWS clients, check: are credentials hardcoded? Are they pulled from environment variables (better but still risky) or from a secrets manager or IAM role (correct)?

VI

Secure Code Review & Threat Modeling

Weeks 24–26

Phase VI builds the two highest-leverage AppSec skills — the ones that appear earliest in a job. Secure code review is the daily work of most AppSec engineers at software companies. Threat modeling is how AppSec influence scales beyond individual code reviews to affect system design. The developer perspective that threat modeling requires — understanding how systems are built and where trust boundaries exist — is built through the pre-study and Phases I–IV before this phase begins. OWASP ASVS is introduced here as the verification standard that gives code review a systematic checklist — turning ad-hoc review into structured, repeatable methodology.

Learning Resources

- **OWASP Code Review Guide v2.0** — The [methodology reference](#) for secure code review.
- **OWASP Application Security Verification Standard (ASVS) v5.0.0** — Approximately 350 testable security requirements across 17 chapters covering authentication, access control, input validation, cryptography, API security, and configuration. Level 1 is the first layer of defense; Level 2 covers standard security practice.
- **“Threat Modeling: Designing for Security”** — Adam Shostack, Wiley, 2014 — The standard text on threat modeling. Ch. 1–3 cover a first threat model, data flow diagrams, and STRIDE; Ch. 4–5 cover attack trees and attack libraries.
- **GitHub Security Lab** — Published CVE research on real open-source projects. Covers vulnerability discovery, exploitation mechanics, and coordinated disclosure.
- **OWASP Threat Dragon** — Threat modeling tool. Used for the threat modeling project in this phase.

Secure Code Review & Threat Modeling — Weeks 24–26

01. READ + PRACTICE: Secure Code Review Methodology: Weeks 24–25

- Read the [OWASP Code Review Guide, Introduction through Reviewing by Vulnerability](#) (methodology, code review techniques, vulnerability-specific review guidance)
- Conduct a scoped code review of a real open-source Python web application (Django, Flask, or FastAPI) from GitHub — not WebGoat, but a real project at real scale. Scope it to one subsystem, for example authentication and session handling, roughly 2,000 lines, and budget about 8 hours
- **Apply the systematic methodology:** trace user input, verify authentication, check authorization, audit crypto, check dependencies

02. READ + PROJECT: Threat Modeling: Weeks 24–26

- Read “[Threat Modeling: Designing for Security](#)” Ch. 1–5
- Build a complete threat model for the API built during pre-study, including the LLM endpoint added in Phase IV. A system built from scratch end-to-end is the right target: every layer is known, every trust boundary can be reasoned about.
- Use [OWASP Threat Dragon](#) for the data flow diagram. Apply STRIDE to each component
- **Produce a threat model document:** DFD, trust boundaries, threat enumeration, risk rating, mitigations

03. STUDY: OWASP ASVS — Verification Standard: Week 26

- Read the [ASVS v5.0.0 Level 1 and Level 2 requirements](#). Do not try to memorize — learn the structure and the categories
- Map each ASVS chapter to the vulnerability classes studied in this plan — input validation and output encoding, authentication, access control, and cryptography each get their own chapter, so you can pin any finding to a specific numbered requirement
- Build a lightweight personal ASVS checklist: 20–30 of the highest-signal Level 2 requirements you would check first in any code review
- This checklist becomes a portfolio artifact: a reusable, standards-backed review tool

04. PRACTICE: Code Review Under Time Pressure: Week 26

- Real code review is time-boxed — a PR waits on you; interview exercises use the same 30–45 minute format
- Use [GitHub Security Lab](#)’s published CVE [writeups](#) as answer keys: read the vulnerable code before the writeup, try to identify the vulnerability, then compare
- Do five of these across different vulnerability classes
- **Target:** 30 minutes from cold code to written vulnerability identification

Projects & Writings — Secure Code Review & Threat Modeling

Project: Open-Source Code Review

Documented security review of a real open-source Python application. Findings in standard format, pushed to GitHub. Includes scope, methodology, findings, and remediation recommendations.

Project: Threat Model Document

Full STRIDE threat model of a system built end-to-end in this plan. DFD, trust boundaries, threat enumeration, risk ratings, mitigations. Pushed to GitHub. An unusual portfolio artifact that shows design-time security thinking, not only exploitation. The target is your Phase 0 FastAPI app including the LLM endpoint added in Phase IV, so STRIDE has to cover prompt injection and output handling as well as the classic web threats.

Checklist: Personal ASVS Review Checklist

20–30 of the highest-signal ASVS Level 2 requirements, ordered as you would check them in a code review. Pushed to GitHub as Markdown; a reusable, standards-backed review tool.

Writing: ‘Threat Modeling the Billing System I Built’ (1,000–1,500 words)

Use your ActiveCampaign billing microservice as the subject. Walk through what a threat model of that system would have looked like, what STRIDE would have identified, and how the architecture would have been

different had AppSec been involved in the design. The most personal piece in the portfolio: a threat model of a system you actually built.

Concepts and Their Use on the Job

Secure code review as methodology

Not reading code and hoping to notice something bad — systematic input tracing, authentication verification, authorization checking, cryptography auditing. Replicable and teachable. **On the job:** When a developer asks you to review a PR, you apply the methodology: trace inputs, check auth, check authz, check crypto, check dependencies. Faster and repeatable when systematic.

OWASP ASVS as a verification framework

~350 testable security requirements organized by domain. Level 1 is the first layer of defense against common attacks. Level 2 is a comprehensive view of standard security practice. Using ASVS during code review means you can say ‘this fails ASVS V2.1.1’ rather than ‘this seems wrong’ — a finding a developer can look up and act on.

STRIDE threat modeling

Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege. A threat category for every component of a data flow diagram. **On the job:** When a team presents a new microservice design, you build a DFD on the whiteboard, identify trust boundaries, and run STRIDE. One threat model at design time can prevent many vulnerability findings after code is written.

Trust boundaries

The lines in a DFD where data crosses from less-trusted to more-trusted context. Every trust boundary is a potential vulnerability surface.

Time-pressured code review

Cold code, 30–45 minutes, a written or spoken identification of the flaw. The same format as a PR review under deadline and as the AppSec interview exercise; practice makes both feel routine.

VII

CompTIA Security+ Certification

Weeks 27–30

CompTIA Security+ is a baseline credential. It is required or preferred as required or preferred in many AppSec job postings, especially at enterprise companies and in regulated industries. It confirms broad security literacy; the portfolio demonstrates hands-on AppSec skill. It is achievable in 3–4 weeks with the foundational knowledge built throughout this plan.

Learning Resources

- [“CompTIA Security+ Get Certified Get Ahead: SY0-701 Study Guide”](#) — Joe Shelley and Darril Gibson, Certification Experts LLC, 2023 — Covers all SY0-701 exam objectives across eleven chapters with real-world examples and exam topic review sections.
- [Professor Messer Practice Exams](#) — Paid practice exams for the Security+ SY0-701 exam. Same format and structure as the actual exam.

CompTIA Security+ Certification — Weeks 27–30

01. READ + MEMORIZE: Study Guide: Weeks 27–30

- Read [“CompTIA Security+ Get Certified Get Ahead: SY0-701 Study Guide”](#) cover to cover. The exam has five domains: General Security Concepts, Threats/Vulnerabilities/Mitigations, Security Architecture, Security Operations, and Security Program Management & Oversight
- Most technical content from the first three domains will be review from earlier phases. Focus study time on governance, compliance, and program management domains — these are the least familiar material
- **Full notes, then granular cards:** label and study note cards in book order — one card per concept (title on the front, 1–5 bullets on the back), roughly 160–200 cards, with extra cards where a topic is dense. Prioritize the high-yield memorization sets: ports and protocols, cryptography and PKI, attack types, security control types, and the governance, compliance, and framework material (your least-familiar domain)
- **Memorize (grouped spaced review):** memorize cards in groups of seven with full front-to-back recall, fold each group into a growing cumulative review pile, and re-drill the whole pile each time you add a new memorized 7 card group to it (pruning only over-learned cards). Budget real time — at ~180 cards this is a multi-day make-and-memorize effort, not an afterthought

02. PRACTICE: Practice Exams: Weeks 29–30

- Take a full [Professor Messer](#) practice exam cold. Score it and identify weak domains
- Study weak domains specifically from the study guide via note cards and additional learning, if required. Retake. Repeat until score 85–90% over 2–3 practice exams in a row

03. EXAM: CompTIA Security+ SY0-701: Week 30

- **Delivery:** schedule through Pearson VUE and take it in person at a [Philadelphia test center](#) (nearest: Temple University, North Broad Street) — about \$439 (2026), valid for three years
- **Scheduling:** book the soonest available seat once you are consistently at 85–90% on practice exams — do not delay and let fresh memorization fade; specific dates fill, so reserve the nearest one the moment you hit target
- **Retakes:** no cap — no wait for the first retake, then a 14-day wait after any further fail, full price each; once you pass you cannot retake the same exam code without CompTIA's consent

04. APPLY: Main Application Push: Week 30 onward

- With Security+ now on your resume, intensify the application push that began in Phase IV — widen to more startups and AppSec-adjacent roles
- **Daily target:** 1–2 applications a day, each with a cover letter tailored to the specific role, the resume tailored as needed, and both checked against an ATS scanner — budget roughly 1–2 hours per application for that personalization
- See “[Application Strategy](#)” in Phase VIII for the full targeting, profile, and framing playbook

Projects & Writings — CompTIA Security+ Certification

Certification: CompTIA Security+ (SY0-701)

Passed exam, valid for three years; the vendor-neutral baseline that confirms shared vocabulary with SOC, infrastructure, and GRC colleagues before AppSec-specific work begins.

Notes: Security+ note-card deck

One card per concept in book order, memorized in grouped spaced review; reused in Phase VIII for interview drilling.

Concepts and Their Use on the Job

Security+ exam domains

All five SY0-701 domains: General Security Concepts, Threats/Vulnerabilities/Mitigations, Security Architecture, Security Operations, Security Program Management & Oversight. **On the job:** Security+ signals baseline security literacy to enterprise companies and regulated industries, where it is a common requirement. The portfolio carries the hands-on evidence.

Governance and compliance

Risk management frameworks, regulatory compliance (HIPAA, PCI-DSS, SOC 2), security policy and procedure, security awareness training. **On the job:** AppSec engineers work with compliance teams. Knowing what PCI-DSS requires for web applications, what HIPAA means for data handling, and how risk frameworks are structured makes cross-team conversations productive.

Cryptography fundamentals

Symmetric and asymmetric encryption, hashing algorithms, PKI, certificate lifecycle, TLS/SSL configuration.

Network security

Firewalls, IDS/IPS, VPN, network segmentation, wireless security.

Identity and access management

Authentication protocols, MFA, SSO, directory services, privileged access management.

Incident response

IR lifecycle phases, forensics fundamentals, log analysis, chain of custody.

Shared vocabulary with the wider security team

Cryptography, network security, and authentication fundamentals from Security+ are the common ground with SOC, infrastructure, and GRC colleagues. The certification confirms that baseline before AppSec-specific work begins.

VIII

Interview Preparation & Interviewing

Weeks 31–34

AppSec interviews test security knowledge, adversarial reasoning, and communication, and rarely algorithm puzzles. This phase runs interview preparation alongside the ongoing job search: rehearsing behavioral answers, drilling technical note cards, practicing the live technical formats — secure code review, light scripting, and system design — and running mock interviews so that the watched, timed setting is familiar before the first real interview.

Learning Resources

- [*“Impractical Python Projects”*](#) — Lee Vaughan, No Starch Press, 2018 — project-based Python practice for writing correct scripts quickly without an IDE's assistance
- [GitHub Security Lab](#) — vulnerability research and published [CVE write-ups](#) used as source material for secure code review practice
- [Security interview question bank](#) — a curated set of cybersecurity interview questions to build a technical note card deck from
- [Mock interview platforms](#) — [Aced \(formerly Exponent\)](#) (security engineering courses) and [interviewing.io](#) (behavioral and observed coding practice)

Interview Preparation & Interviewing — Weeks 31–34

01. REHEARSE: Behavioral Questions: Weeks 31–34

- Write out full answers to the standard behavioral questions, memorize them, and rehearse them aloud until your delivery sounds natural rather than recited.
- Rehearse AppSec-specific prompts as well: handling a developer who disputes a finding, prioritizing a backlog of vulnerabilities, explaining risk to a non-technical stakeholder, and so forth.

02. MEMORIZE: Technical Note Cards: Weeks 31–34

- Build note cards organized by topic: the OWASP Top 10 and the individual web vulnerability classes, the Security+ domains, PortSwigger attack techniques, and defensive and secure-coding patterns.
- Alongside cards drawn from your own study, create a deck from a reputable existing set of cybersecurity interview questions at [a curated public question bank](#), rewriting the answers in your own words.
- Use the same grouped spaced review as Phase VII step 01: groups of seven, cumulative review pile, periodic re-reads of finished topics.

03. PRACTICE: Secure Code Review: Weeks 31–34

- Practice a clear, spoken routine until it becomes automatic: read the code, name the vulnerability class, explain how it is exploited, and recommend the fix.
- Work through some of [GitHub Security Lab](#)'s published CVE [write-ups](#) on unfamiliar code, at about five reviews each week.
- Practice naming the vulnerability class immediately from a code sample — for example: user input passed into a query without sanitizing it (injection); a React component using `dangerouslySetInnerHTML` (cross-site scripting); a JSON Web Token whose signature algorithm is accepted as 'none' (authentication bypass); a user identifier read from a request parameter with no authorization check (insecure direct object reference); or a URL fetched by the server without validation (server-side request forgery).

04. PRACTICE: Light Scripting & System Design: Weeks 31–34

- Refresh your ability to write correct Python quickly without IDE hints; some companies include a short coding session (ex. CoderPad) — a small script to parse a log, fix a function, or automate a check, rather than an algorithm puzzle. [Impractical Python Projects](#) is good practice, and your development background covers most of this.
- Practice the security system-design format: sketch a small architecture — for example the trust boundaries of an authentication flow or a microservice — mark where data crosses each trust boundary, apply the STRIDE model to enumerate the threats, and rehearse walking through it aloud. This is verbal whiteboard practice, not a written deliverable.

05. PRACTICE: Mock Interviews: Weeks 31–34

- Run at least one full mock interview each week, covering behavioral questions and a code review, through [Aced \(formerly Exponent\)](#) (its security engineering courses) or [interviewing.io](#), so that a real interview is never the first time you perform under observation.
- Build up on your own first — untimed, then timed, then recorded and reviewed against your own answers — before booking the weekly mock interviews above.
- When you do not immediately know an answer, narrate your reasoning aloud and state what you do know and how you would investigate it, so that a pause becomes a spoken plan rather than silence.

Application Strategy

- **Start applying at Week 16:** security-conscious startups are viable targets from Week 16 with Phases I–III complete, BSCP preparation under way, thirteen published writeups (the SQL injection essay, the JWT article, the ten OWASP Top 10 pieces, and the SSRF writeup), and PortSwigger Academy progress visible. Do not wait for curriculum completion.
- **Target the titles that fit an entry:** Security Engineer I or II, Product Security Engineer, Junior Application Security Engineer, and DevSecOps Engineer, alongside Application Security Engineer. Many postings with the last title ask for three or more years of security experience; five years of production development plus this portfolio is relevant experience, so apply.
- **Pin your best AppSec work:** set your GitHub pinned repositories to the key new artifacts — the [Juice Shop assessment report](#), your [custom Semgrep rule](#), and the [threat model](#) — so a reviewer sees application-security work first, not the older web-dev projects.
- **Make LinkedIn application-ready:** list BSCP and Security+ under licenses and certifications as each is passed; link your portfolio projects and published write-ups; and carry the application-security transition narrative in the About section. Publish as you work through the plan, so you have work to discuss at local cyber security community meetups.
- **Frame the transition:** in cover letters and interviews, lead with the narrative — five years of full-stack software engineering, then a deliberate move into application security in August 2026. Point to this plan and portfolio as the evidence.

IX

Expected First Days on the Job

Week 35+

You will arrive knowing the vulnerability classes and tooling. You will be slower on internal tooling, company-specific security policies, and the organizational dynamics of working with development teams at varying security maturity. The plan budgets 4–8 weeks for that ramp.

First Two Weeks

Read the existing security documentation the way you read a codebase — understand what is there before suggesting what should change. Ask about the current vulnerability backlog before proposing new tooling. Understand what development teams build and how they ship before inserting yourself into their process.

- What is the current SAST setup, if any? What do developers receive as output?
- What does the existing pentest methodology look like? How are findings tracked?
- Who are the key development team contacts? What are their current security concerns?
- What does the incident response process look like? Has it been tested?

The question to ask in the first week is not ‘what should we add?’ It is ‘what is already here, and why is it the way it is?’ The answer tells you everything about the organizational constraints you are working within.

Your Immediate Differentiators

Developer empathy

You understand why developers write vulnerable code because you wrote it too. That shared experience makes conversations with development teams direct and credible.

API security depth

REST API security is a central AppSec domain. Hands-on API security knowledge built throughout this plan means arriving on day one prepared for the work.

Automation instinct

Semgrep rule development, CI/CD integration, and tooling scripting reuse skills from engineering work.

Full-stack deployment

You can own the deployment of security tooling from writing the Semgrep rule to integrating it into GitHub Actions to triaging the results.

The Gap and How Long It Takes to Close

The distance between ‘knows the vulnerability classes and tooling’ and ‘productive AppSec team member who moves fast with developers’ is the ramp this plan budgets 4–8 weeks for. You already speak the language of the teams you will be working with. You will be a junior with a steep learning curve ahead.

Appendix

Reference Material · Philadelphia AppSec Community

Philadelphia AppSec Community

Security networking operates differently from general tech networking. The community is smaller, more tight-knit, and more willing to help people who show genuine curiosity. Attending events before you are employed builds relationships with local practitioners. Show up, ask good questions, and bring something to demonstrate — even a completed PortSwigger lab that taught you something surprising.

Core Groups

- **OWASP Philadelphia Chapter** (events on [Meetup](#) · Free) — The most directly relevant group for AppSec networking in Philadelphia. OWASP (Open Web Application Security Project) is the professional organization for web application security globally. The Philadelphia chapter holds regular meetings with technical presentations on vulnerability classes, tooling, and real-world AppSec work. The people in this room are AppSec practitioners at companies across the region — exactly the referral network you are building toward.
- **DC215 — Philadelphia DEF CON Group** (events on [Meetup](#) · Free) — DEF CON Groups are local offshoots of DEF CON, the largest hacker conference in the world. DC215 holds regular meetups with technical talks, CTF practice, and informal knowledge sharing. Builder-focused and practitioner-dense. The format rewards people who bring something to show. Attend when Phase III work (vulnerability classes) is underway.
- **BSides Philadelphia** (Annual · December 11, 2026 · \$80 early bird) — Community-organized security conference with talks across offensive, defensive, and AppSec tracks. Affordable, full-day, attended by local practitioners from the companies you will apply to. Volunteer if possible — it is how you get face time with organizers and speakers.
- **Philly 2600** (First Friday monthly · Free) — Philadelphia's oldest hacker meetup — running since the early 1990s and one of the first 2600 meetings ever listed in 2600 Magazine. Meets the first Friday of every month at Iffy Books starting at 6pm. No set agenda, no presentations, no gatekeeping — a casual hangout where people bring things to hack on or show off. Open to anyone. Around 8pm the group typically moves to a nearby bar. The lowest-pressure entry point into the Philly security community and a good place to show up early in the plan before you have polished work to present.
- **PhillySec** (Second Wednesday monthly · Free) — Monthly security meetup, second Wednesday of each month. More structured than Philly 2600 — presentations and technical discussions rather than open hangout format. Good mid-plan target once Phase II or III is underway and you have something to contribute to conversations.
- **OWASP Delaware Chapter** (Wilmington, DE · Monthly · Free) — The OWASP chapter an hour south. Runs a monthly MetaCTF flash CTF night, hands-on exercises, and secure-coding presentations. A second AppSec-specific room when the Philadelphia calendar is quiet.
- **Ex Machina Parlor** (319 N 11th St, Room 5D · Philadelphia) — A hackerspace specifically aimed at people interested in cybersecurity. Workshop and collaborative hacking space rather than a meetup. Worth

knowing it exists as a physical space to work alongside practitioners. Check their calendar for events and open nights.

Conferences

- **OWASP Global AppSec USA** (Annual · autumn, rotating US city; 2026 is San Francisco) — The flagship AppSec-specific conference in the US, with tracks for builders, breakers, defenders, and OWASP projects. This is the single most targeted conference in the plan — the audience is exactly the community you are entering and the talks directly map to the skills in every phase. Attend once Phase IV or V is complete.
- **OWASP BASC — Boston** (Annual · April) — The OWASP Boston Application Security Conference. Regional, practitioner-dense, focused on AppSec engineering and secure development. Lower-cost than Global AppSec USA. Strong emphasis on hands-on content and OWASP standards — the audience overlaps heavily with who you will work alongside in the role.
- **Thotcon — Chicago** (next event June 11–12, 2027) — A mid-sized hacker conference: more intimate than DEF CON, technically dense, practitioner-focused. Falls after this plan ends; a target for your first year in the role.
- **JawnCon — Glenside, PA** (Annual · October) — Philadelphia's own security and technology conference, held at Arcadia University just outside the city. Covers a wide range of security and tech topics with a deliberately approachable, community feel. Worth attending as a local conference where you will run into the same practitioners from OWASP and DC215.
- **BSides Delaware — Newark, DE** (Annual · November 13–14, 2026 · \$25) — The nearest BSides after Philadelphia, at the University of Delaware, about an hour away. Talks, villages, and a CFP; the same regional practitioner crowd as BSides Philly. Falls in Week 14, so a realistic first conference with Phase III work in hand.
- **DEF CON — Las Vegas** (Annual · August) — the flagship hacker conference. Within it, the **AppSec Village** is a 100% volunteer-run space specifically for application security — attacker-minded talks, live demos, hands-on workshops, and an AppSec-focused CTF, the most directly relevant part of DEF CON for this career path. A larger, costlier trip to Las Vegas than the regional events — treat it as a target for a future year, once you are established in the field, rather than something to attend during this plan.